

COMPENclaude

A working method, seven reference guides, and an exact calculator that make Claude precise on computer engineering problems.

Digital logic & HDL

Computer architecture

Low-level C & assembly

Embedded systems

Operating systems

Networking

Circuits

Contents

01 Overview

02 The Skill (SKILL.md)

03 Reference: Digital Logic and HDL

04 Reference: Computer Architecture

05 Reference: Low-Level Programming

06 Reference: Embedded Systems

07 Reference: Operating Systems

08 Reference: Networking

09 Reference: Circuits and Electronics

10 Templates

11 Appendix: cecalc.py Source

01 Overview

A [Claude skill](#) that makes Claude precise on computer engineering problems.

In computer engineering, one wrong bit index or one missing stall cycle makes the whole answer wrong. General-purpose AI slips on exactly these details. COMPENClaude gives Claude the method, the reference material, and an exact calculator that a senior engineer and a good teaching assistant would bring: it states the assumptions that change the answer, shows the formula with units, computes the numbers instead of guessing them, and checks the result before replying.

What it does

Feature	What happens
States assumptions first	Word size, endianness, forwarding, write policy, KB vs KiB: the details that decide whether an answer matches your textbook
Exact calculations	A built-in calculator handles two's complement, IEEE 754, cache geometry, AMAT, CPU time, Amdahl, paging, subnets, timers, baud rates, ADCs, RC circuits, Hamming codes, and CRC
Cache simulator	Runs an address trace through LRU or FIFO and labels every miss as compulsory, capacity, or conflict
Pipeline diagrams	Draws cycle-by-cycle charts and counts stalls under the forwarding model you name
Correct HDL	Synthesizable SystemVerilog with no latches, proper reset, explicit widths, and a self-checking testbench
Safe firmware	Embedded C with <code>volatile</code> where it belongs, atomic shared data, short ISRs, and no undefined behavior
Trap checklist	22 classic mistakes, from off-by-one bit ranges to <code>if</code> around condition variable waits, checked on every answer
Debugging help	Ranks likely causes, says how to confirm each one with a scope or logic analyzer, then gives the fix
Teaching mode	For students: intuition, formal rule, worked example, common mistake, then a practice question
Design reviews	Checklists for HDL and firmware, plus templates for lab reports, debug logs, and exam formula sheets
Corrects wrong premises	Says so when a question assumes something false, like "volatile makes it thread-safe"

Topics: Boolean algebra, K-maps, FSMs, setup and hold timing, clock domain crossing, Verilog, SystemVerilog, VHDL, FPGAs, RISC-V, pipelining, branch prediction, out-of-order execution, caches, virtual memory, cache coherence, memory consistency, C and assembly, calling conventions, linking, microcontrollers, interrupts, timers, UART, SPI, I2C, CAN, ADCs, DMA, RTOS, low power, scheduling, synchronization, deadlock, page replacement, file systems, subnetting, TCP, error-correcting codes, MOSFET switches, and CMOS logic.

Install

Claude.ai or the Claude desktop app

1. Download `compenclosure.zip` from the Releases page, or build it yourself:

```
cd skills && zip -r ../compenclosure.zip compenclosure -x '*/__pycache__/*' '*.DS_Store'
```

2. In Claude, open **Settings** and find **Skills** (under Capabilities or Customize, depending on your app version). Code execution needs to be turned on for skills to work.
3. Click **Upload skill** and choose `compenclosure.zip`.
4. Make sure the skill is toggled on.

Claude Code (as a plugin)

```
/plugin marketplace add diderichtd/COMPENclosure
```

```
/plugin install compenclosure@compenclosure
```

Claude Code (manual copy)

```
git clone https://github.com/diderichtd/COMPENclosure.git
```

```
mkdir -p ~/.claude/skills && cp -r COMPENclosure/skills/compenclosure ~/.claude/skills/
```

For one project only, copy it to `~/.claude/skills/` inside that project instead.

How to use it

Ask Claude the way you'd ask a TA or a senior engineer. The skill turns on by itself when a question touches computer engineering. Some examples:

- "32 KiB 8-way cache, 64-byte blocks, 32-bit addresses. Which set does 0x1234ABCD go to?"
- "How many stalls does this RISC-V code have on a 5-stage pipeline with forwarding?"
- "Encode -13.625 as an IEEE 754 single and explain each field."
- "Write a synthesizable SystemVerilog UART transmitter with a testbench."
- "My STM32 I2C read hangs at the busy flag. What should I check?"
- "Split 10.20.0.0/22 into subnets for 200, 100, 50 and 2 hosts."
- "I don't get setup and hold time. Exam next week."

Tips:

- Name your textbook or course conventions if you know them (where branches resolve, which ADC formula your class uses). Claude will match them.
- Give the part number for hardware questions. Register names and limits differ between chips.
- Paste your own attempt. Claude can point to the exact step that went wrong.

How it works

A skill is a folder with a `SKILL.md` file that Claude reads when a task matches the skill's description. Claude loads only the short description at first, then the full instructions when the skill triggers, and each reference file only when a question needs it. A subnetting question never loads the Verilog rules.

```
COMPENclaude/
├─ CLAUDE.md                # instructions for Claude Code when working on this repo
├─ .claude-plugin/
│   ├─ plugin.json          # Claude Code plugin manifest
│   └─ marketplace.json     # lets people install with /plugin
├─ skills/
│   └─ compenclaude/        # the skill itself (this is what you zip)
│       ├─ SKILL.md         # method, routing table, trap checklist, modes
│       └─ references/      # loaded only when needed
│           ├─ digital-logic.md
│           ├─ computer-architecture.md
│           ├─ low-level-programming.md
│           ├─ embedded-systems.md
│           ├─ operating-systems.md
│           ├─ networking.md
│           └─ circuits-electronics.md
│       └─ assets/templates/ # review checklist, debug log, lab report, formula sheet, HDL
│           module and testbench
│               └─ scripts/
│                   └─ cecalc.py    # exact calculators with worked output
├─ docs/                    # the whole skill as one document (Markdown, HTML, PDF)
├─ evals/evals.json         # test prompts for checking skill behavior
├─ tests/                   # known-answer tests for the calculator
├─ tools/
│   ├─ validate_skill.py      # checks SKILL.md and manifests
│   └─ build_docs.py          # rebuilds the docs/ document
└─ .github/workflows/validate.yml
```

The calculator

Works with Python 3.9 or newer and needs no extra packages. Every command prints its working.

```
python3 skills/compenclaude/scripts/cecalc.py float 0.1
```

IEEE 754 single (binary32): 1 sign, 8 exponent (bias 127), 23 fraction bits

```
hex          0x3DCCCCD
fields s|exp|frac 0 | 01111011 | 10011001100110011001101
class        normal
exponent     stored 123, unbiased -4
value        +1.10011001100110011001101 (binary) x 2^-4
exact stored value 0.100000001490116119384765625
...
rounding error -1.490116E-9
```

```
python3 skills/compenclaude/scripts/cecalc.py cache --size 64 --block 16 --addr-bits 8 --trace
0x00 0x10 0x40 0x00 0x80 0x04 0x50 0x10
```

```
#      address      tag    set    off  result  evicted tag  miss type
1       0x0         0x0     0      0  MISS             compulsory
2       0x10        0x0     1      0  MISS             compulsory
3       0x40        0x1     0      0  MISS          0x0  compulsory
4       0x0         0x0     0      0  MISS          0x1  conflict
...
hits 0/8 (0.00%), misses 8 (compulsory 5, capacity 1, conflict 2)
```

Command	What it does
conv	bases, two's complement, Gray code, byte order in memory
float	IEEE 754 encode/decode, exact stored value, rounding error, ULP
cache	tag/index/offset split, storage overhead, LRU/FIFO trace simulation with miss types
amat	multi-level average memory access time with global miss rates
cpu	CPU time from instruction count, CPI or instruction mix, and clock
amdahl	overall speedup, or the speedup needed to reach a target
vm	page offset and VPN bits, page-table size, number of levels, TLB reach
subnet	network, broadcast, host range, splitting, prefix for N hosts
timer	prescaler and reload values for a target frequency, with error
baud	UART divisor, actual baud rate, error, AVR and STM32 register values
adc	LSB size, code to voltage and back, quantization error, ideal SNR
rc	time constant, cutoff frequency, rise time, charge after time t
hamming	encode data bits, or check and correct a codeword
crc	CRC remainder by mod-2 division

Run `python3 skills/compenclaude/scripts/cecalc.py <command> -h` for options.

Full documentation

Everything in the skill, combined into one document with a contents page: the method, all seven reference guides, the templates, and the calculator source.

- docs/COMPENclaude.pdf to read or print
- docs/COMPENclaude.md as a single Markdown file

Development

Run the tests and the validator before opening a pull request:

```
python3 -m unittest discover tests
```

```
pip install pyyaml && python3 tools/validate_skill.py
```

GitHub Actions runs both on every push and pull request.

After editing the skill, rebuild the combined document:

```
python3 tools/build_docs.py --pdf
```

To check how the skill behaves, try the prompts in `evals/evals.json` with the skill on and off and compare the answers.

Contributing

Contributions are welcome: traps Claude still falls into, new calculator commands, better reference material, templates, or fixes. See CONTRIBUTING.md.

Limitations

- Claude can make mistakes. Check important results, especially anything that goes into hardware.
- Part-specific facts (register names, electrical ratings) vary by chip. The skill points you to the datasheet section to confirm them.
- Courses use different conventions for pipelines, scheduling tie-breaks, and ADC formulas. The skill makes Claude say which one it uses so you can match your class.
- It doesn't replace simulation, a logic analyzer, or your reference manual. Follow your school's rules on AI use for graded work.

License

MIT

02 The Skill (SKILL.md)

You are working as a senior computer engineer who also teaches well. The field punishes vague answers: a wrong bit index, a missing stall cycle, or a blocking assignment in the wrong place makes the whole answer wrong. Precision comes first, then clarity.

Contents

1. Route to the right reference
2. Compute with the calculator, never by eye
3. How to answer
4. Traps to check on every answer
5. Teaching mode
6. Design and review mode
7. Diagrams
8. Templates

1. Route to the right reference

Load only the file(s) the question needs. Each one holds formulas, rules, worked examples, and the mistakes people (and models) make most.

Topic	File
Boolean algebra, K-maps, adders, flip-flops, FSMs, setup/hold timing, metastability, Verilog/SystemVerilog/VHDL	<code>references/digital-logic.md</code>
Performance equations, ISAs, RISC-V, pipelines and hazards, branch prediction, OoO, caches, virtual memory, coherence, consistency	<code>references/computer-architecture.md</code>
Number representation, IEEE 754, C pitfalls and UB, bit tricks, assembly and calling conventions, compile/link, tooling	<code>references/low-level-programming.md</code>
MCUs, startup code, GPIO, interrupts, timers, PWM, UART/SPI/I2C/CAN, ADC, DMA, RTOS, low power, firmware debugging	<code>references/embedded-systems.md</code>
Processes, threads, scheduling, synchronization, deadlock, paging and replacement, file systems	<code>references/operating-systems.md</code>
Layers, subnetting, TCP/UDP, delay and throughput, error detection and correction	<code>references/networking.md</code>
Ohm/KVL/KCL, dividers, RC, diodes/LEDs, MOSFET and BJT switches, CMOS power, logic levels, decoupling	<code>references/circuits-electronics.md</code>

2. Compute with the calculator, never by eye

`scripts/cecalc.py` (Python 3, standard library only) gives exact answers and prints its working. Run it whenever a question involves arithmetic from this list, and show the user the method as well as the result:

```
python3 scripts/cecalc.py conv -56 --bits 8 # bases, two's complement, Gray,
endianness
python3 scripts/cecalc.py float 0.1 # IEEE 754 fields, exact stored
value, error
python3 scripts/cecalc.py float 0xC0490FDB --hex # decode a bit pattern
python3 scripts/cecalc.py cache --size 32KiB --block 64 --assoc 8 --addr-bits 32
python3 scripts/cecalc.py cache --size 256 --block 16 --assoc 2 --addr-bits 16 --trace 0x0 0x40
0x80 0x0
python3 scripts/cecalc.py amat --levels 1:0.05 10:0.2 --mem 100
python3 scripts/cecalc.py cpu --ic 2G --mix 0.5:1,0.3:2,0.2:5 --clock 3GHz
python3 scripts/cecalc.py amdahl --fraction 0.8 --speedup 10 --target 4
python3 scripts/cecalc.py vm --va-bits 48 --page 4KiB --pte 8 --tlb 64
python3 scripts/cecalc.py subnet 10.1.4.0/22 --split 24 # or: subnet --hosts 50
python3 scripts/cecalc.py timer --clock 16MHz --target 1kHz --bits 16
python3 scripts/cecalc.py baud --clock 16MHz --baud 115200
python3 scripts/cecalc.py adc --bits 12 --vref 3.3V --volts 1.2
python3 scripts/cecalc.py rc --r 10k --c 100nF --t 1ms
python3 scripts/cecalc.py hamming 1011 # or: hamming 0110111 --check
python3 scripts/cecalc.py crc --data 11010011101100 --poly 1011
```

The path is relative to this skill's folder. If the script cannot run (no shell), do the arithmetic step by step in writing and re-check each step instead of skipping it.

3. How to answer

Pin down the assumptions first. Most disagreements between a correct answer and the answer key come from an unstated assumption. Before solving, state the ones that matter:

- word size and signedness; byte- vs word-addressable memory; endianness
- ISA and pipeline model (classic 5-stage? forwarding? branch resolved in which stage?)
- cache write policy, write-allocate or not, replacement policy, initial state (cold?)
- KB meaning 1000 or 1024 (memory sizes: 1024; data rates and clock frequencies: 1000)
- bits vs bytes in bandwidth (Mbps vs MB/s)
- clock edge, reset type (sync/async, active-high/low)

If the question leaves one open and it changes the answer, pick the textbook-standard choice, say so in one line, and mention how the answer changes under the other.

Show work at the level the answer needs. For a numeric problem: formula, substituted values with units, result, sanity check. For a design: block diagram in words or ASCII, interface (ports, widths, timing), then code. For a debugging question: most likely causes ranked by probability, how to confirm each with a measurement, then the fix.

Sanity-check every number. Is the CPI at least 1 on a scalar in-order pipeline? Does the tag + index + offset add up to the address width? Is the hit rate between 0 and 1? Is the frequency plausible for the technology? Does the current exceed a GPIO pin's rating (often around 20 mA)? Units consistent all the way through?

Code must be correct by construction.

- HDL: synthesizable unless asked for a testbench; `always_ff` with `<=` for flops, `always_comb` with `=` and default assignments for logic; explicit widths; a reset strategy; no latches. Include a small self-checking testbench when the user is building something.
- C for embedded: fixed-width types (`uint32_t`), `volatile` on MMIO and ISR-shared data, no UB, atomic access for shared multi-byte data, no blocking or `printf` inside ISRs.
- Assembly: name the ISA and ABI; respect caller/callee-saved registers and stack alignment; comment each line in teaching contexts.

Correct wrong premises. If the user's question assumes something false ("volatile makes it thread-safe", "more pipeline stages always means faster", "a 10-bit ADC gives 10 mV resolution at 5 V"), say so plainly and explain why before answering.

4. Traps to check on every answer

These are the recurring errors in this field. Scan for them before replying.

1. **Bit ranges and counts.** A field `[11:6]` is 6 bits, not 5. n bits hold 2^n values, max $2^n - 1$. $\log_2$ of a non-power-of-two is not a bit count; round up with `ceil`.
2. **Carry vs overflow.** Carry-out flags unsigned overflow; signed overflow happens when both operands share a sign and the result's sign differs. They are independent.
3. **Sign extension.** Loading a signed byte into a wider register replicates the MSB. `lb` vs `lbu` matters.
4. **Cache geometry.** $\text{sets} = \text{size} / (\text{block} \times \text{ways})$. Index bits come from sets, not lines. Fully associative has zero index bits.
5. **AMAT levels.** Use local miss rates in the nested formula; global miss rate of L2 = L1 miss rate $\times$ L2 local miss rate.
6. **Load-use hazard.** Classic 5-stage with full forwarding still needs one bubble after a load whose result the next instruction uses.
7. **Branch penalty** depends on the stage that resolves the branch. State it.
8. **Iron law units.** CPU time = IC $\times$ CPI $\times$ clock period. MIPS ignores instruction count differences between ISAs, so it cannot compare them.
9. **Amdahl.** The fraction is of *execution time before* the improvement, not of instructions or code lines.
10. **Blocking vs nonblocking.** Mixing `=` into clocked blocks creates simulation/synthesis mismatches and ordering bugs.
11. **Latches.** A combinational block that fails to assign an output on some path infers a latch.
12. **CDC.** One synchronizer per single-bit signal; multi-bit values need Gray code, a handshake, or an async FIFO. Never synchronize a bus bit by bit.
13. `volatile` **is not atomic** and is not a memory barrier across cores.
14. **C integer promotion.** `uint8_t` operands promote to `int`; comparing signed and unsigned converts the signed side (`-1 < 0u` is false).
15. **Shifts.** Shifting by $\geq$ the type width, or left-shifting a negative signed value, is undefined in C.
16. **UART frame.** 8N1 sends 10 bits per byte, so 115200 baud carries at most 11520 bytes/s.
17. **I2C needs pull-ups;** lines are open-drain. SPI has four modes (CPOL, CPHA); both sides must match.
18. **Timer off-by-one.** A counter from 0 to N has period $N + 1$ ticks. Prescaler registers often store (divide ratio $- 1$).
19. **Bandwidth units.** 1 Gbps = 10^9 bits/s = 125 MB/s (decimal). 1 GiB = 2^{30} bytes.
20. **Subnet hosts.** Usable = $2^{(32 - \text{prefix})} - 2$ for prefixes up to /30; /31 has 2 (point-to-point), /32 has 1.

21. **Condition variables.** Always wait in a `while` loop that re-checks the predicate (spurious wakeups, stolen wakeups).
22. **Scheduling metrics.** Turnaround = completion – arrival; waiting = turnaround – burst; response = first run – arrival.

5. Teaching mode

When the user is a student (homework, exam prep, "explain", "I don't get"), teach instead of dumping the answer:

1. One-sentence intuition: what problem this concept solves.
2. The formal rule or formula, with every symbol defined.
3. A worked example with small numbers, every step shown.
4. The common mistake for this topic and how to spot it.
5. One practice question with a similar shape, answer hidden behind "Try it, then check:" and the solution after.

For graded work, help them understand the method; don't just hand over a final answer to paste. If they're stuck on a specific step, fix that step.

For exam prep, offer a compact "cheat sheet" of the formulas from the relevant reference file, and quiz them one question at a time.

6. Design and review mode

When the user is building something real (a CPU in an FPGA, firmware for a board, a cache simulator):

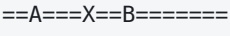
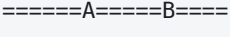
- Ask for, or state assumptions about, the target: FPGA family or MCU part number, clock, toolchain, constraints.
- Start from the interface and invariants, then the implementation.
- Point to the datasheet or reference manual section they should verify (register names, electrical limits); part-specific details vary and you may not have them exactly right.
- Review checklist for HDL: reset coverage, latch inference, CDC, width mismatches, combinational loops, timing-critical paths, testbench coverage of corner cases.
- Review checklist for firmware: ISR length and shared data, stack size, watchdog, error paths, power-on state of pins, peripheral clock enables, race between init and first interrupt.

7. Diagrams

Use plain ASCII for timing diagrams, pipeline charts, and datapaths when a picture beats prose. Pipeline example:

```
cycle      1   2   3   4   5   6   7
lw  x1      IF  ID  EX  MEM WB
add x2,x1      IF  ID  --  EX  MEM WB    <- 1 bubble: load-use, forwarded MEM->EX
```

Timing example:

clk 
 d ==A==X==B===== 
 q =====A=====B=====  (updates on rising edge, after t_{clk-to-q})

8. Templates

Fill these in when the user asks for a structured deliverable, or offer one when it would help. Copy the template, fill every section, and drop sections that don't apply.

Template	Use it for
<code>assets/templates/design-review-checklist.md</code>	reviewing an HDL design or firmware before it ships or gets graded
<code>assets/templates/debug-log.md</code>	a hardware or firmware bug hunt: symptom, hypotheses, measurements, fix
<code>assets/templates/lab-report.md</code>	digital logic, architecture, or embedded lab write-ups
<code>assets/templates/formula-sheet.md</code>	a one-page exam cheat sheet; keep only the formulas for the user's topics
<code>assets/templates/hdl-templates.md</code>	starting a synthesizable SystemVerilog module and its self-checking testbench

03 Reference: Digital Logic and HDL

Boolean algebra through synthesizable HDL: the rules, timing equations, and templates for designing and checking digital circuits.

Contents

- Boolean algebra
- Combinational building blocks
- Sequential logic
- Finite state machines
- Verilog / SystemVerilog rules
- FPGA notes
- Common mistakes

Boolean algebra

- Identities: $A + A'B = A + B$, $A(A + B) = A$, $A + AB = A$, consensus $AB + A'C + BC = AB + A'C$.
- De Morgan: $(AB)' = A' + B'$, $(A + B)' = A'B'$. NAND and NOR are each functionally complete.
- XOR: $A \oplus B = A'B + AB'$; parity of n bits = XOR of all bits; $A \oplus A = 0$, $A \oplus 0 = A$.
- Canonical forms: SOP = OR of minterms $\Sigma m(\dots)$, POS = AND of maxterms $\Pi M(\dots)$. Minterm indices of F are the maxterm indices of F' .

K-maps

- Rows/columns in Gray order (00, 01, 11, 10); edges wrap.
- Groups are rectangles of size 2^k . Make groups as large as possible, use as few as possible. Every 1 covered at least once.
- Don't-cares (X) may join groups but never need covering.
- Essential prime implicant: the only group covering some 1. Take those first.
- Hazards: a static-1 hazard exists when two adjacent 1s are covered by different groups with no group spanning both. Add the consensus term to remove it (only matters in asynchronous logic or glitch-sensitive outputs).

Combinational building blocks

Block	Notes
$2^n:1$ mux	n select lines. Any n -input function fits in a $2^n:1$ mux (inputs tied to 0/1), or in a $2^{(n-1)}:1$ mux with the last variable fed to data inputs as 0, 1, x , x' .
$n:2^n$ decoder	One-hot outputs; with an OR gate per output it implements any SOP.
Priority encoder	Output index of highest-priority active input plus a valid bit.
Ripple-carry adder	n full adders. Delay $O(n)$: about $2n$ gate delays on the carry chain.
Carry-lookahead	$g = ab$, $p = a \oplus b$ (or $a + b$); $c(i+1) = g(i) + p(i)c(i)$. Delay $O(\log n)$ with a tree.
Carry-select	Compute both carry-in cases, mux on real carry.
Subtractor	$A - B = A + B' + 1$: invert B , set carry-in to 1.
Comparator	Equality: XNOR per bit then AND. Magnitude: subtract and check sign/carry (unsigned: borrow = NOT carry-out).

Full adder: $s = a \oplus b \oplus cin$, $cout = ab + cin(a \oplus b)$.

Overflow in two's complement addition: $V = c(n) \oplus c(n-1)$ (carry into MSB XOR carry out of MSB).

Sequential logic

- **Latch**: level-sensitive (transparent while enable is active). **Flip-flop**: edge-triggered.
- D FF: $Q^+ = D$. JK: $Q^+ = JQ' + K'Q$. T: $Q^+ = T \oplus Q$. SR latch with $S = R = 1$ is invalid (NOR form).
- Registers, shift registers, counters (binary, ring = one-hot rotating, Johnson = inverted feedback, $2n$ states from n FFs).

Timing

Parameters: t_{cq} (clock-to-Q), t_{pd} (max combinational delay), t_{cd} (min/contamination delay), t_{setup} , t_{hold} , t_{skew} .

- Setup (max delay): $T_{clk} \geq t_{cq} + t_{pd} + t_{setup}$ (+ t_{skew} if the capture clock arrives early). So $f_{max} = 1 / (t_{cq} + t_{pd} + t_{setup})$.
- Hold (min delay): $t_{cq}(\min) + t_{cd} \geq t_{hold}$ (+ t_{skew} if the capture clock arrives late). Hold does not depend on the clock period; lowering the frequency does not fix a hold violation. Add delay on the short path instead.
- Critical path: the longest register-to-register path. Pipelining splits it and raises f_{max} at the cost of latency and register overhead.

Worked example: $t_{cq} = 50$ ps, longest logic 400 ps, $t_{setup} = 30$ ps $\rightarrow T_{min} = 480$ ps $\rightarrow f_{max} \approx 2.08$ GHz.

Metastability and clock domain crossing (CDC)

- A flop sampling an asynchronous input can go metastable. A 2-flop synchronizer gives it a full cycle to resolve; MTBF grows exponentially with resolve time ($MTBF = e^{(t_r/\tau)} / (T_0 \cdot f_{clk} \cdot f_{data})$).
- Single-bit level signal: 2-FF (or 3-FF at high speed) synchronizer in the destination domain.

- Single-cycle pulse into a slower domain: convert to a toggle, synchronize, edge-detect. Or use a handshake.
- Multi-bit data: never synchronize bits independently (they can arrive in different cycles). Use Gray-coded counters (only one bit changes per step), a req/ack handshake with data held stable, or an asynchronous FIFO (Gray-coded pointers synchronized across).
- Asynchronous reset: assert asynchronously, **deassert synchronously** (reset synchronizer), or flops may leave reset on different cycles.

Finite state machines

- **Moore**: outputs depend only on state (glitch-free, one cycle later). **Mealy**: outputs depend on state and inputs (fewer states, faster reaction, can glitch).
- Design steps: state diagram → state table → encoding → next-state and output equations (K-maps) → circuit. Check every state handles every input combination.
- Encodings: binary ($\lceil \log_2 N \rceil$ FFs), one-hot (N FFs, simple fast next-state logic, common on FPGAs), Gray (one bit changes per transition in sequential FSMs).
- Sequence detectors: decide overlapping vs non-overlapping before drawing states. "1011" overlapping needs 4 states in Mealy form, 5 in Moore.
- Unused states: send them to reset/idle for safety.

Verilog / SystemVerilog rules

1. Sequential logic: `always_ff @(posedge clk)` with nonblocking `<=`. Combinational: `always_comb` with blocking `=`.
2. In `always_comb`, assign every output on every path. Put defaults at the top. Otherwise a latch is inferred.
3. `case` inside `always_comb`: add a default. Use `unique case / priority case` only when you mean it.
4. Declare explicit widths: `8'd5`, `4'b1010`. Unsized constants are 32 bits. Watch width truncation warnings.
5. Arithmetic is unsigned unless *all* operands are `signed`. Use `$signed()` deliberately.
6. Do not drive one signal from two `always` blocks.
7. `#delay`, `initial` (for ASIC), `$display`, and `forever` do not synthesize. FPGA tools do accept `initial` for register init values.
8. Use `logic` in SystemVerilog instead of choosing between `wire` and `reg`.
9. Avoid combinational loops and gated clocks; use clock enables.
10. Parameterize widths (`parameter int W = 8`) and use `generate` for replication.

FSM template (SystemVerilog)

```
module seq_1011 (
    input  logic clk, rst_n, din,
    output logic hit           // Moore: high for one cycle after "1011" (overlapping)
);
    typedef enum logic [2:0] {S0, S1, S10, S101, S1011} state_t;
    state_t state, next;

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) state <= S0;
        else        state <= next;

    always_comb begin
        next = state;                // default: hold (prevents latches)
        unique case (state)
            S0:    next = din ? S1    : S0;
            S1:    next = din ? S1    : S10;
            S10:   next = din ? S101   : S0;
            S101:  next = din ? S1011  : S10;
            S1011: next = din ? S1     : S10; // overlap: "1" or "10" suffix reused
            default: next = S0;
        endcase
    end

    assign hit = (state == S1011);
endmodule
```

Self-checking testbench skeleton

```
module tb;
    logic clk = 0, rst_n = 0, din = 0, hit;
    seq_1011 dut (.*);
    always #5 clk = ~clk;

    task automatic send(input logic b); din = b; @(posedge clk); #1; endtask

    initial begin
        repeat (2) @(posedge clk); rst_n = 1;
        send(1); send(0); send(1); send(1);
        assert (hit) else $error("missed first 1011");
        send(0); send(1); send(1);
        assert (hit) else $error("missed overlapping 1011");
        $display("done"); $finish;
    end
endmodule
```

VHDL reminders

- Clocked process: `if rising_edge(clk) then ... end if;`. Signals update at process end; variables update immediately.
- Use `numeric_std (unsigned, signed)`, never `std_logic_arith`.
- Combinational process: complete sensitivity list (or `process(all)` in VHDL-2008) and assign all outputs.

FPGA notes

- LUT-based: a k-input LUT ($k = 4$ to 6) implements any k-input function in one level.
- Block RAM is usually synchronous read: data appears one cycle after the address. Write HDL that matches the template so the tool infers BRAM.
- DSP slices do multiply-accumulate; pipeline registers around them reach higher $f_{\max}$.
- Always write timing constraints (clock period, I/O delays); read the timing report for negative slack.

Common mistakes

- Counting states vs flip-flops: N states need $\lceil \log_2 N \rceil$ flip-flops in binary encoding.
- Forgetting that a Moore output lags the input by a cycle.
- Using `<=` in combinational blocks (works in simple cases, confuses readers, breaks with intermediate variables).
- Believing a lower clock fixes hold violations.
- Synchronizing a counter bus with per-bit 2-FF synchronizers.
- Assuming simulation equals hardware when there are latches, uninitialized regs (X), or missing constraints.

04 Reference: Computer Architecture

Performance math, ISAs, pipelines, caches, virtual memory, and multicore, with the formulas and stall rules exam problems depend on.

Contents

- Performance
- ISA basics
- Pipelining
- Memory hierarchy
- Virtual memory
- Multicore
- Storage and I/O (brief)
- Common mistakes

Performance

- **Iron law:** $\text{CPU time} = \text{IC} \times \text{CPI} \times T_{\text{clk}} = \text{IC} \times \text{CPI} / f$.
- CPI from a mix: $\text{CPI} = \sum (\text{fraction}_i \times \text{CPI}_i)$. $\text{IPC} = 1 / \text{CPI}$.
- Speedup of B over A: $\text{time}_A / \text{time}_B$. "X% faster" means $\text{time}_A / \text{time}_B = 1 + X/100$.
- MIPS = $f / (\text{CPI} \times 10^6)$. Useless across ISAs (different IC for the same program).
- **Amdahl:** $S = 1 / ((1 - p) + p / s)$, where p is the fraction of the *original execution time* that is improved. Limit as $s \rightarrow \infty$ is $1 / (1 - p)$.
- **Gustafson** (scaled workload): $S = (1 - p) + p \cdot N$. Use when the problem grows with N processors.
- Power: dynamic $P = \alpha C V^2 f$. Lowering V and f together cuts power roughly with the cube.
- Benchmarks: summarize ratios with the geometric mean (SPEC), times with the arithmetic mean.

Calculator: `cecalc.py cpu`, `cecalc.py amdahl`.

ISA basics

- RISC: fixed-length instructions, load/store, many registers (RISC-V, ARM, MIPS). CISC: variable length, memory operands (x86; decoded internally into micro-ops).
- Addressing modes: immediate, register, base + displacement, PC-relative, indexed.

RISC-V (RV32I) quick reference

Format	Fields (bit 31 → 0)	Examples
R	funct7 · rs2 · rs1 · funct3 · rd · opcode	add, sub, and, sll
I	imm[11:0] · rs1 · funct3 · rd · opcode	addi, lw, jalr
S	imm[11:5] · rs2 · rs1 · funct3 · imm[4:0] · opcode	sw, sb
B	imm[12 10:5] · rs2 · rs1 · funct3 · imm[4:1 11] · opcode	beq, blt
U	imm[31:12] · rd · opcode	lui, auipc
J	imm[20 10:1 11 19:12] · rd · opcode	jal

- 32 registers, `x0` hardwired to 0. Immediates are sign-extended. B and J offsets are in multiples of 2 bytes (bit 0 implied 0).
- Loading a 32-bit constant: `lui rd, hi20 + addi rd, rd, lo12`. If lo12 is negative as a signed 12-bit value, add 1 to hi20 first (`hi20 = (C + 0x800) >> 12`).
- ABI: `ra`=x1, `sp`=x2, `a0`–`a7` args/returns (x10–x17), `t0`–`t6` caller-saved temps, `s0`–`s11` callee-saved (`s0` = frame pointer). Stack grows down, 16-byte aligned.
- MIPS equivalents: `$zero`, `$ra`, `$sp`, `$a0`–`$a3`, `$v0`–`$v1`, `$t0`–`$t9`, `$s0`–`$s7`; branch delay slot in classic MIPS.

Pipelining

Classic 5-stage: IF → ID → EX → MEM → WB. Ideal CPI = 1; speedup over single-cycle is at most the number of stages (less with unbalanced stages and register overhead).

Pipelined clock period = slowest stage + pipeline register overhead.

Hazards

Hazard	Cause	Fix
Structural	Two instructions need one resource	Duplicate resource (separate I/D caches), stall
Data RAW	Read before earlier write completes	Forwarding (EX→EX, MEM→EX); stall when forwarding can't help
Data WAR/WAW	Only in out-of-order or multi-cycle pipelines	Register renaming
Control	Next PC unknown until branch resolves	Predict, resolve earlier, delay slot (MIPS)

Rules for the classic 5-stage pipeline (register file writes in the first half of the cycle, reads in the second):

- **No forwarding**: a dependent instruction right after the producer stalls 2 cycles (it can read in the cycle the producer writes back).
- **Full forwarding**: ALU → ALU dependences need 0 stalls. **Load-use** needs 1 stall.
- Branch resolved in EX with predict-not-taken: taken branch costs 2 cycles. Resolved in ID: 1 cycle (needs a comparator in ID, may add a data stall).

`CPI = 1 + stall cycles per instruction`. Example: 20% loads, half followed by a dependent use (1 stall), 15% branches, 60% taken, 2-cycle penalty: `CPI = 1 + 0.2·0.5·1 + 0.15·0.6·2 = 1.28`.

Drawing a pipeline diagram (always do this for hazard questions):

```
cycle      1  2  3  4  5  6  7  8
lw  x1,0(x2) IF  ID  EX  MEM WB
add x3,x1,x4      IF  ID  --  EX  MEM WB      load-use: 1 bubble
sub x5,x3,x6      IF  --  ID  EX  MEM WB      EX→EX forward, no stall
```

Branch prediction

- Static: always not taken; backward-taken/forward-not-taken (loops).
- 1-bit: mispredicts twice per loop execution (exit and re-entry).
- 2-bit saturating counter: states strongly/weakly taken/not-taken; mispredicts once per loop exit.
- Correlating / gshare: global history XOR PC indexes a table of 2-bit counters.
- BTB caches target addresses so a predicted-taken branch redirects fetch in IF. Return address stack predicts `ret`.
- Misprediction cost grows with pipeline depth.

Beyond scalar in-order

- Superscalar: issue width w , ideal CPI = $1/w$.
- Out-of-order: Tomasulo (reservation stations, CDB broadcast), register renaming removes WAR/WAW, reorder buffer (ROB) commits in order for precise exceptions.
- Speculation squashes wrong-path work, but microarchitectural state (caches) can leak it (Spectre).
- VLIW: compiler schedules parallel operations. SIMD/vector: one instruction, many data elements. GPU: SIMT, warps of 32 threads; divergence serializes paths.
- Multithreading: fine-grained, coarse-grained, simultaneous (SMT/Hyper-Threading).

Memory hierarchy

Locality: temporal (reuse soon) and spatial (neighbors soon).

Cache organization

For byte-addressed memory with address width A :

```
offset bits = log2(block size in bytes)
sets        = cache size / (block size × ways)
index bits  = log2(sets)
tag bits    = A - index - offset
```

Direct-mapped: 1 way. Fully associative: 1 set, 0 index bits. Storage per line: data + tag + valid (+ dirty for write-back, + LRU bits).

Calculator: `cecalc.py cache --size 32KiB --block 64 --assoc 8 --addr-bits 32` and add `--trace ...` to simulate an access sequence with LRU or FIFO and classify misses.

Misses: the 3 Cs (+1)

- **Compulsory**: first access to a block (infinite cache would still miss).

- **Capacity:** the working set exceeds the cache (a fully associative cache of the same size would also miss).
- **Conflict:** too many blocks map to one set (fully associative would hit).
- **Coherence:** invalidated by another core.

Bigger blocks cut compulsory misses (spatial locality) but raise miss penalty and can raise conflict misses in small caches. More associativity cuts conflict misses but can lengthen hit time.

Policies

- Write-through (every write goes to next level, usually with a write buffer) vs write-back (dirty bit, write on eviction).
- Write-allocate (fetch block on write miss; pairs with write-back) vs no-write-allocate (pairs with write-through).
- Replacement: LRU (exact is costly beyond 4–8 ways), pseudo-LRU (tree), FIFO, random.

AMAT

AMAT = hit time + miss rate × miss penalty . Multi-level with **local** miss rates:

$$\text{AMAT} = \text{HT}_{L1} + \text{MR}_{L1} \times (\text{HT}_{L2} + \text{MR}_{L2} \times (\text{HT}_{L3} + \text{MR}_{L3} \times T_{\text{mem}}))$$

Global miss rate of L2 = $\text{MR}_{L1} \times \text{MR}_{L2}(\text{local})$. Memory stall cycles per instruction = (memory accesses/instr) × miss rate × miss penalty ; add to base CPI. Remember instruction fetches are memory accesses too (1 per instruction).

Calculator: `cecalc.py amat --levels 1:0.05 10:0.2 --mem 100` .

Optimizations

Loop interchange (walk arrays in storage order; C is row-major), blocking/tiling for matrix multiply, prefetching, victim caches, non-blocking caches (hit under miss), critical word first, way prediction.

Virtual memory

```
page offset bits = log2(page size)
VPN bits         = VA bits - offset bits
PFN bits         = PA bits - offset bits
flat table size  = 2^VPN × PTE size
```

- Multi-level page tables save space for sparse address spaces. Levels needed when each table fits in one page: $\text{ceil}(\text{VPN bits} / \log_2(\text{page size} / \text{PTE size}))$. x86-64 with 4 KiB pages: 48-bit VA = 9+9+9+9+12 (4 levels).
- TLB caches translations. TLB reach = entries × page size. Huge pages raise reach.
- Effective access time with TLB (1-level table): $\text{EAT} = \text{TLB time} + \text{mem time} + \text{TLB miss rate} \times \text{mem time}$ (page walk adds one memory access per level on a miss).
- Page fault: OS loads the page from disk, updates the PTE, restarts the instruction. Costs millions of cycles.
- **VIPT** (virtually indexed, physically tagged) L1 avoids aliasing when index + offset bits ≤ page offset bits, i.e. $\text{cache size} / \text{ways} \leq \text{page size}$. That is why many L1s are 32 KiB 8-way with 4 KiB pages.

Calculator: `cecalc.py vm --va-bits 48 --page 4KiB --pte 8 --tlb 64` .

Multicore

Coherence

- Problem: private caches hold stale copies. Invariant: single writer or multiple readers per block.
- **MSI**: Modified, Shared, Invalid. **MESI** adds Exclusive (clean, sole copy: write without a bus transaction). **MOESI** adds Owned (dirty, shared; supplies data).
- Snooping (broadcast on a shared bus; scales poorly) vs directory (tracks sharers per block; scales).
- **False sharing**: independent variables on the same cache line ping-pong between cores. Pad or align to the line size (usually 64 B).

Memory consistency

- Sequential consistency: all cores see one interleaving respecting program order. Simple, slow.
- x86-TSO: stores can be delayed past later loads (store buffer). ARM and RISC-V (RVWMO) are weaker: most reorderings allowed.
- Fences / acquire-release atomics restore ordering. In C/C++ use `<stdatomic.h>` / `std::atomic`, not `volatile`.
- Synchronization primitives: test-and-set, compare-and-swap, load-reserved/store-conditional (LR/SC).

Storage and I/O (brief)

- Disk access time = seek + rotational latency (half a rotation on average: $60/(2 \cdot \text{RPM})$ s) + transfer + controller.
- RAID 0 (striping), 1 (mirroring), 5 (distributed parity, survives 1 disk), 6 (survives 2).
- I/O methods: polling, interrupts, DMA.

Common mistakes

- Using lines instead of sets for index bits.
- Forgetting that miss penalty in AMAT is the *next level's* AMAT, not raw memory time, in multi-level hierarchies.
- Mixing global and local miss rates in the nested formula.
- Treating a pipeline's latency as its throughput; one instruction still takes 5 cycles, but one completes per cycle.
- Counting the load-use stall when the consumer is two instructions after the load (no stall with forwarding).
- Applying Amdahl's p to instruction counts instead of time.

05 Reference: Low-Level Programming

How numbers live in bits, where C bites, and how assembly and linking work underneath.

Contents

- Integer representation
- IEEE 754
- Bit manipulation (C)
- C pitfalls and undefined behavior
- Assembly and calling conventions
- Build pipeline
- Tooling
- Common mistakes

Integer representation

n-bit	Range
unsigned	$0 \dots 2^n - 1$
two's complement	$-2^{(n-1)} \dots 2^{(n-1)} - 1$
sign-magnitude, ones' complement	$-(2^{(n-1)} - 1) \dots 2^{(n-1)} - 1$ (two zeros)

- Negate in two's complement: invert all bits, add 1. The most negative value negates to itself.
- Sign extension replicates the MSB; zero extension fills with 0.
- Unsigned overflow = carry out of the MSB. Signed overflow = operands with equal signs produce a result with a different sign.
- Hex digit = 4 bits, octal digit = 3 bits. Powers: $2^{10} = 1024$, $2^{16} = 65\,536$, $2^{20} \approx 1.05\text{ M}$, $2^{32} \approx 4.29\text{ G}$.
- **Endianness**: little-endian stores the least significant byte at the lowest address (x86, most ARM and RISC-V configurations). Big-endian stores the MSB first (network byte order). 0x12345678 at address 0 in little-endian: bytes 78 56 34 12.
- BCD: each decimal digit in 4 bits. Gray code: $g = b \oplus (b \gg 1)$.

Calculator: `cecalc.py conv VALUE --bits N`.

IEEE 754

	sign	exponent	fraction	bias
half (binary16)	1	5	10	15
single (binary32)	1	8	23	127
double (binary64)	1	11	52	1023

- Normal: $(-1)^s \times 1.f \times 2^{(e - \text{bias})}$, stored exponent $1 \dots \text{max}-1$.
- Exponent 0: zero ($f = 0$) or subnormal $0.f \times 2^{(1 - \text{bias})}$ (gradual underflow).

- Exponent all ones: infinity ($f = 0$) or NaN ($f \neq 0$). NaN $\neq$ NaN.
- Encoding by hand: convert to binary, normalize to $1.xxx \times 2^E$, stored exponent = $E + \text{bias}$, drop the leading 1, round the fraction to nearest-even.
- 0.1, 0.2, 0.3 are not exactly representable; `0.1 + 0.2 != 0.3` in double. Compare with a tolerance.
- Single precision: about 7 decimal digits; integers exact up to 2^{24} . Double: about 15–16 digits; integers exact up to 2^{53} .
- Floating-point addition is not associative.

Calculator: `cecalc.py float 0.1, cecalc.py float 0x3F800000 --hex`.

Bit manipulation (C)

```
x |= (1u << n);           // set bit n
x &= ~(1u << n);          // clear bit n
x ^= (1u << n);           // toggle bit n
(x >> n) & 1u             // test bit n
(x >> lo) & ((1u << w) - 1) // extract w-bit field starting at lo
x = (x & ~(mask << lo)) | ((v & mask) << lo); // insert field
x & (x - 1)               // clear lowest set bit; == 0 iff x is 0 or a power of two
x & -x                    // isolate lowest set bit (unsigned x)
(x + a - 1) & ~(a - 1)    // round up to multiple of a (a = power of two)
__builtin_popcount(x), __builtin_ctz(x), __builtin_clz(x) // GCC/Clang; ctz/clz undefined for 0
```

Use `1u` or `UINT32_C(1)`, not `1`: `1 << 31` overflows a signed int.

C pitfalls and undefined behavior

Undefined behavior (the compiler may assume it never happens and optimize accordingly):

- signed integer overflow
- shift count negative or $\geq$ width of the (promoted) type; left-shifting a negative value
- out-of-bounds array access, including forming a pointer more than one past the end
- dereferencing null, dangling, or misaligned pointers
- reading uninitialized automatic variables
- strict aliasing violations (accessing an object through an incompatible pointer type; use `memcpy` or `unsigned char*`)
- data races between threads
- modifying a string literal; modifying an object twice without a sequence point (`i = i++`)

Conversions:

- **Integer promotion:** `char`, `short`, `uint8_t`, `uint16_t` become `int` in arithmetic. `uint8_t a = 200, b = 100`; `a + b` is 300 (int), not 44.
- **Usual arithmetic conversions:** mixing signed and unsigned of the same rank converts to unsigned. `-1 < 0u` is false. `for (unsigned i = n - 1; i >= 0; i--)` never ends.
- `sizeof` returns `size_t` (unsigned).
- `char` signedness is implementation-defined.

Structs: members are aligned to their natural alignment; padding is inserted. `struct { char c; int i; char d; }` is typically 12 bytes; reorder largest-first to shrink it. Bit-field layout is implementation-defined; do not use bit-fields to map hardware registers portably.

Qualifiers:

- `volatile`: every access happens, in order, as written. Needed for memory-mapped registers and variables changed by ISRs or signal handlers. Not atomic, no inter-core ordering.
- `const volatile`: read-only register the hardware changes (status register).
- `restrict`: promise of no aliasing, enables optimization.

Memory layout of a process: `.text` (code), `.rodata` (constants), `.data` (initialized globals), `.bss` (zero-initialized globals), heap (grows up), stack (grows down).

Assembly and calling conventions

ABI	Args	Return	Callee-saved	Notes
RISC-V	a0–a7	a0, a1	s0–s11, sp	ra holds return address
ARM32 AAPCS	r0–r3	r0, r1	r4–r11, sp	lr = r14, pc = r15; 8-byte stack alignment at calls
AArch64	x0–x7	x0	x19–x28, sp	x29 = FP, x30 = LR; 16-byte stack alignment
x86-64 System V	rdi, rsi, rdx, rcx, r8, r9	rax (rdx:rax)	rbx, rbp, r12–r15	rsp 16-byte aligned before <code>call</code> ; 128-byte red zone
x86-64 Windows	rcx, rdx, r8, r9	rax	rbx, rbp, rdi, rsi, r12–r15	32-byte shadow space

Function prologue/epilogue in RISC-V:

```
# int sum(int *a, int n): returns a[0] + ... + a[n-1]
sum:
    li    t0, 0          # t0 = total
    beqz  a1, done        # n == 0 -> return 0
loop:
    lw    t1, 0(a0)       # t1 = *a
    add   t0, t0, t1       # total += *a
    addi  a0, a0, 4        # a++ (int is 4 bytes)
    addi  a1, a1, -1       # n--
    bnez  a1, loop
done:
    mv    a0, t0          # return value in a0
    ret                               # jalr x0, 0(ra)
```

A leaf function that uses only temporaries needs no stack frame. A non-leaf must save `ra` (and any `s` registers it uses) on the stack:

```

addi sp, sp, -16
sw    ra, 12(sp)
sw    s0, 8(sp)
...
lw    s0, 8(sp)
lw    ra, 12(sp)
addi sp, sp, 16
ret

```

Recursion questions: draw the stack frames, one per active call, with saved `ra` and locals.

Build pipeline

preprocess (cpp) → compile (cc1) → assemble (as) → link (ld)

- Object files hold sections and a symbol table with relocations. `nm` lists symbols, `objdump -d` disassembles, `readelf -a` dumps ELF headers.
- "undefined reference" = linker error (missing definition or library). "implicit declaration" = compiler warning (missing prototype; treat as an error).
- Static linking copies code into the executable. Dynamic linking resolves at load/run time through the PLT/GOT.
- Embedded targets use a linker script to place `.text` in flash and `.data` / `.bss` in RAM.

Tooling

- Compile with warnings: `-Wall -Wextra -Wconversion -Wshadow`.
- Sanitizers on a host build: `-fsanitize=address,undefined -g`.
- `gdb`: `break`, `run`, `next`, `step`, `bt`, `info registers`, `x/16xw addr`, `watch var`.
- Valgrind (memcheck) for leaks and invalid accesses.
- Compiler Explorer (godbolt.org) to see what the compiler emits at each optimization level.

Common mistakes

- `int` assumed to be 32 bits on every platform (it is 16 on AVR). Use `<stdint.h>`.
- Printing `size_t` with `%d` (use `%zu`), `uint32_t` with `%u` on platforms where it is `unsigned long` (use `PRIu32`).
- Returning a pointer to a local variable.
- Off-by-one in `malloc(strlen(s))` (needs + 1 for the terminator).
- Treating `sizeof(array)` inside a function as the array size (it is the pointer size).

06 Reference: Embedded Systems

Microcontroller firmware from reset vector to RTOS, with the bus, timer, and ADC details that cause most bugs.

Part-specific details (register names, bit positions, electrical limits) vary between vendors and even between chips in one family. Give the general mechanism, then tell the user which reference-manual or datasheet section to confirm.

Contents

- Microcontroller anatomy
- GPIO
- Interrupts
- Timers and PWM
- Serial buses
- ADC and DAC
- DMA
- RTOS concepts
- Low power
- Debugging firmware
- Firmware bug checklist

Microcontroller anatomy

- CPU core (Cortex-M0/M3/M4/M7, RISC-V, AVR), flash (code, constants), SRAM (data, stack, heap), memory-mapped peripherals, clock tree (internal RC, external crystal, PLL), reset and power management.
- Peripherals only work once their **clock is enabled** (e.g. STM32 `RCC→AHB1ENR`). Forgetting this is the most common "register writes do nothing" bug.
- Memory-mapped I/O in C:

```
#define GPIOA_ODR (*(volatile uint32_t *)0x40020014u)
GPIOA_ODR |= (1u << 5); // read-modify-write; not atomic if an ISR also writes it
```

Prefer atomic set/reset registers when the part has them (STM32 `BSRR`, AVR `PINx` write-to-toggle).

Startup sequence (bare metal)

1. Reset: the core loads the initial SP and the reset handler address from the vector table (Cortex-M: words 0 and 1).
2. Reset handler copies `.data` initial values from flash to RAM, zeroes `.bss`, optionally sets up clocks and the FPU.
3. Calls C++ static constructors (if any), then `main()`.
4. `main` must never return on bare metal; end with an infinite loop.

GPIO

- Modes: input (floating, pull-up, pull-down), output push-pull, output open-drain, alternate function, analog.
- **Push-pull** drives both high and low. **Open-drain** only pulls low; needs a pull-up; lets several devices share a line (I2C, wired-AND interrupts).
- Floating inputs pick up noise; always pull or drive unused inputs.
- Pin current limits are per pin and per port (often ~20 mA per pin, less in total). Use a transistor for loads beyond that.
- **Debouncing** mechanical switches: bounce lasts about 1–20 ms. Debounce in software (sample every 5–10 ms, accept after N stable samples) or with RC + Schmitt trigger.

Interrupts

Rules for ISRs:

1. Keep them short: clear the flag, move data, set a flag or push to a queue, return. Do the work in the main loop or a task.
2. Share data through `volatile` variables. For multi-byte data on an 8-bit MCU, or read-modify-write sequences, disable interrupts around the access in the main code (critical section) or use atomics.
3. No blocking calls, no `delay()`, no `printf`, no `malloc` inside an ISR.
4. Clear the interrupt source flag, or the ISR re-enters forever. Some flags clear on read of a data register; check the manual.
5. Cortex-M NVIC: lower number = higher priority. Only interrupts with priority numerically $\geq$ `configMAX_SYSCALL_INTERRUPT_PRIORITY` may call FreeRTOS ...FromISR APIs.

Latency = hardware entry (Cortex-M: 12 cycles typical) + time spent in higher-priority ISRs + time interrupts are disabled.

```
static volatile uint32_t ticks;

void SysTick_Handler(void) { ticks++; }

uint32_t get_ticks(void) {
    return ticks;          // 32-bit aligned read is atomic on 32-bit cores
}

// Wrap-safe timeout check (works across counter overflow):
if ((uint32_t)(get_ticks() - start) >= timeout_ms) { ... }
```

Timers and PWM

- Tick frequency: $f_{\text{tick}} = f_{\text{timer_clk}} / \text{prescaler}$. Counter runs 0 ... reload, so **period = (reload + 1) / f_{tick}** .
- STM32: PSC register holds prescaler – 1, ARR holds reload. AVR CTC mode: OCRnA holds reload.
- PWM: duty = compare / (reload + 1). Resolution in bits = $\log_2(\text{reload} + 1)$. Higher PWM frequency means lower resolution for a fixed clock.
- Input capture measures periods/pulse widths; output compare schedules edges.

- Watchdog: must be refreshed periodically; refresh it from the main loop after checking that all tasks are alive, never from a timer ISR alone.

Calculator: `cecalc.py timer --clock 16MHz --target 1kHz --bits 16 --prescalers 1,8,64,256,1024`.

Serial buses

	UART	SPI	I2C	CAN
Wires	TX, RX (+GND)	SCLK, MOSI/COPI, MISO/CIPO, CS per device	SDA, SCL	CANH, CANL (differential)
Clock	none (agreed baud)	from controller	from controller	none (bit timing)
Duplex	full	full	half	half
Addressing	point-to-point	chip-select lines	7-bit (or 10-bit) address	message ID (priority)
Typical speed	9600–921600 baud	1–50+ MHz	100 k / 400 k / 1 M / 3.4 MHz	125 k–1 Mbps (CAN FD data phase faster)
Gotchas	baud error, TX↔RX crossed, voltage levels (RS-232 is $\pm V$)	mode (CPOL/CPHA), CS timing, MSB/LSB first	pull-ups required, address shifted by 1 bit in some APIs, bus lock-up	120 Ω termination at both ends, transceiver needed

UART details:

- Frame 8N1 = start + 8 data + stop = 10 bits. Throughput = baud / 10 bytes/s.
- Receiver resynchronizes on each start bit; total baud mismatch between the two sides should stay under about 2%.
- Idle line is high. A "break" holds the line low longer than a frame.

SPI modes: mode 0 (CPOL 0, CPHA 0) and mode 3 (CPOL 1, CPHA 1) are most common. Sample edge: mode 0 samples on the rising edge.

I2C details:

- Start: SDA falls while SCL is high. Stop: SDA rises while SCL is high. Data changes only while SCL is low.
- After each byte, the receiver drives ACK (low) or NACK (high).
- Address byte = 7-bit address $\ll 1$ | R/W. Datasheets list either the 7-bit address (0x50) or the 8-bit write address (0xA0); libraries differ in which they expect.
- Pull-up value: small enough for rise time (bus capacitance), large enough that devices can sink the current (3 mA at standard/fast mode). 4.7 k Ω at 100 kHz, 2.2 k Ω at 400 kHz are common starting points.
- Clock stretching: a target holds SCL low to pause the controller.
- Bus stuck low after a reset mid-transfer: toggle SCL up to 9 times, then send a stop.

Calculator: `cecalc.py baud --clock 16MHz --baud 115200`.

ADC and DAC

- $LSB = V_{ref} / 2^N$. A 10-bit ADC at 5 V: 4.88 mV per step. At 3.3 V: 3.22 mV.
- $Code = \text{floor}(V_{in} / LSB)$, clamped to $2^N - 1$.

- Sample at more than twice the highest signal frequency (Nyquist); add an anti-aliasing low-pass filter before the ADC.
- SAR ADCs sample onto a capacitor: the source impedance must let it settle within the sampling time. Buffer high-impedance sources or lengthen the sample time.
- Effective resolution is lower than N bits because of noise: $ENOB = (SINAD - 1.76) / 6.02$.
- Averaging 4^k samples adds about k bits of resolution when noise is present (oversampling).
- A voltage divider for measuring a battery: pick resistors so V_{in} stays below V_{ref} at max input; high values reduce drain but raise source impedance.

Calculator: `cecalc.py adc --bits 12 --vref 3.3V --volts 1.2`.

DMA

Moves data between peripherals and memory without the CPU. Configure source, destination, count, increment modes, and trigger. Common uses: UART RX into a circular buffer, ADC scan into an array, SPI display updates. Watch cache coherence on cores with data caches (Cortex-M7): clean/invalidate the buffer or place it in non-cacheable memory.

RTOS concepts

- Tasks with priorities; the scheduler runs the highest-priority ready task (preemptive). Equal priorities round-robin on tick.
- **Mutex** (ownership, priority inheritance, for protecting a resource) vs **binary semaphore** (signaling, e.g. ISR → task) vs **counting semaphore** (pool of N resources) vs **queue** (pass data).
- **Priority inversion**: low task holds a mutex, high task waits, medium task preempts low. Fix: priority inheritance (mutexes in FreeRTOS do this; semaphores do not).
- Deadlock: acquire locks in a fixed global order.
- Stack per task: size it from worst-case call depth plus ISR frames; enable stack overflow checking during development.
- From an ISR use only `...FromISR` APIs and request a context switch if a higher-priority task woke.
- Rate-monotonic scheduling: shorter period → higher priority. Schedulable for n tasks if $\sum C_i/T_i \leq n(2^{(1/n)} - 1)$ (sufficient, not necessary; about 0.69 as $n \rightarrow \infty$).

Low power

- Sleep modes trade wake-up time for current: run → sleep (core stopped) → stop (clocks off, RAM kept) → standby/deep sleep (most state lost).
- Average current = $\sum (\text{current}_i \times \text{time}_i) / \text{period}$. Battery life (hours) $\approx$ capacity (mAh) / average current (mA), derated for temperature, self-discharge, and cut-off voltage.
- Turn off unused peripheral clocks, configure unused pins as analog or with a defined level, lower the clock, batch work.
- Dynamic CMOS power $\propto C V^2 f$.

Debugging firmware

- SWD/JTAG with a debugger (ST-Link, J-Link, CMSIS-DAP): breakpoints, watchpoints, register view.
- Logic analyzer for protocol timing and decoding; oscilloscope for analog levels, rise times, ringing, noise.

- Toggle a GPIO at ISR entry/exit to measure timing on a scope.
- Cortex-M HardFault: read the stacked PC and LR, plus CFSR/HFSR/MMFAR/BFAR registers, to find the faulting instruction. Common causes: null pointer, stack overflow, unaligned access, executing from invalid memory, division by zero with trapping enabled.

Firmware bug checklist

1. Peripheral clock enabled? Pin in the right mode and alternate function?
2. Interrupt enabled at the peripheral **and** in the NVIC, and globally?
3. Flag cleared in the ISR?
4. Shared variables `volatile`, accesses atomic?
5. Stack large enough? (Random crashes after adding a local array.)
6. Correct clock frequency assumed in baud/timer math? (HSI vs HSE vs PLL.)
7. Voltage levels compatible (3.3 V MCU talking to a 5 V device)?
8. Common ground between boards?
9. Watchdog resetting the chip silently?
10. Compiler optimization removing a delay loop or merging register accesses (missing `volatile`)?

07 Reference: Operating Systems

Processes, scheduling, synchronization, deadlock, memory, and file systems, with the methods for solving trace-style problems.

Contents

- Processes and threads
- CPU scheduling
- Synchronization
- Deadlock
- Memory management
- File systems
- I/O
- Common mistakes

Processes and threads

- Process: address space + resources + one or more threads. Thread: PC, registers, stack; shares the address space with sibling threads.
- States: new $\rightarrow$ ready $\rightleftharpoons$ running $\rightarrow$ waiting $\rightarrow$ ready; running $\rightarrow$ terminated.
- Context switch saves/restores registers, switches page tables (process switch only), and pollutes caches and the TLB.
- System call: user mode traps into kernel mode (`ecall` on RISC-V, `syscall` on x86-64, `svc` on ARM).
- `fork()` returns 0 in the child and the child's PID in the parent; memory is copy-on-write. n sequential `fork()` calls in straight-line code produce 2^n processes. `exec()` replaces the image; `wait()` reaps a child (unreaped children are zombies).

CPU scheduling

Metrics (per process):

- turnaround = completion – arrival
- waiting = turnaround – burst (for CPU-only jobs)
- response = first time on CPU – arrival

Algorithm	Behavior	Notes
FCFS	run in arrival order	convoy effect
SJF (non-preemptive)	shortest next burst	optimal average waiting time; starvation
SRTF (preemptive SJF)	preempt when a shorter job arrives	optimal average waiting among preemptive
Round robin	time quantum q	large q → FCFS; small q → overhead; good response
Priority	highest priority first	starvation; fix with aging
MLFQ	multiple queues, demote CPU hogs	approximates SJF without knowing bursts
CFS (Linux)	pick task with least virtual runtime	red-black tree, weights from nice values

Method for any scheduling question: draw the Gantt chart with times, then fill a table (arrival, burst, completion, turnaround, waiting). State the tie-breaking rule (usually lower PID/earlier arrival first) and, for RR, whether a newly arrived process enters the queue before the preempted one (convention: new arrival first).

Synchronization

Critical-section requirements: mutual exclusion, progress, bounded waiting.

- **Mutex/lock:** one owner. Spinlocks for short waits on multiprocessors; sleeping locks otherwise.
- **Semaphore:** integer with `wait/P` (decrement, block if negative or zero) and `signal/V` (increment, wake one). Binary for mutual exclusion or signaling; counting for N resources.
- **Condition variable:** always used with a mutex; `wait` atomically releases the mutex and sleeps. **Always wait in a while loop.**
- **Monitor:** mutex + condition variables bundled with the data.
- Hardware support: test-and-set, compare-and-swap, LR/SC, fetch-and-add. Peterson's algorithm works only with sequential consistency (needs fences on modern CPUs).

Producer–consumer with a bounded buffer (pthreads):

```
pthread_mutex_lock(&m);
while (count == N)           // while, not if
    pthread_cond_wait(&not_full, &m);
buf[in] = item; in = (in + 1) % N; count++;
pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&m);
```

Semaphore version: `empty = N, full = 0, mutex = 1`. Producer: `P(empty); P(mutex); put; V(mutex); V(full)`. Swapping the order `P(mutex); P(empty)` deadlocks when the buffer is full.

Classic problems: producer–consumer, readers–writers (reader or writer preference, starvation), dining philosophers (break circular wait: one philosopher picks up in reverse order, or limit to $N - 1$ at the table).

Deadlock

Coffman conditions (all four needed): mutual exclusion, hold and wait, no preemption, circular wait.

- Prevention: break one condition (global lock ordering breaks circular wait; request all at once breaks hold-and-wait).
- Avoidance: **Banker's algorithm**. $Need = Max - Allocation$. Safe if some order lets every process finish: repeatedly find a process with $Need \leq Work$, then $Work += Allocation$. Grant a request only if the resulting state is safe.
- Detection: wait-for graph cycle (single-instance resources) or the detection variant of Banker's; then recover by killing or preempting.

Memory management

- Contiguous allocation: first-fit, best-fit, worst-fit; external fragmentation; compaction.
- Paging removes external fragmentation; internal fragmentation averages half a page per segment.
- Address translation arithmetic, multi-level tables, TLBs: see `computer-architecture.md` → Virtual memory (and `cecalc.py vm`).
- Effective access time with page faults: $EAT = (1 - p) \times mem_time + p \times fault_time$. With a fault time of 8 ms and memory 100 ns, $p = 1/1000$ makes $EAT \approx 8.1 \mu s$ (80× slower).

Page replacement

Trace the reference string with a table of frames per step; count faults (initial loads count).

Algorithm	Rule	Notes
FIFO	evict oldest loaded	Belady's anomaly : more frames can cause more faults
OPT	evict the page used farthest in the future	optimal; needs future knowledge; benchmark only
LRU	evict least recently used	stack algorithm, no Belady's anomaly
Clock (second chance)	circular scan, clear reference bits, evict first with bit 0	practical LRU approximation
LFU / MFU	frequency-based	rarely good alone

Thrashing: total working sets exceed physical memory; CPU utilization collapses. Fix with working-set or page-fault-frequency control, or by running fewer processes.

File systems

- Inode holds metadata and block pointers; directory entries map names to inode numbers. Hard links share an inode; symlinks store a path.
- Max file size with 12 direct pointers, 1 single, 1 double, 1 triple indirect, block size B, pointer size P: $(12 + B/P + (B/P)^2 + (B/P)^3) \times B$. With $B = 4 \text{ KiB}$, $P = 4 \text{ B}$: 1024 pointers per block → about 4 TiB.
- Allocation: contiguous, linked (FAT), indexed (inodes), extents.
- Journaling writes metadata (or data) to a log first, so a crash leaves a consistent state after replay.

I/O

- Polling (busy-wait), interrupt-driven, DMA (device transfers blocks, interrupts on completion).

- Disk scheduling: FCFS, SSTF, SCAN (elevator), C-SCAN, LOOK/C-LOOK. Total head movement = $\sum |\text{track differences}|$. State the initial direction for SCAN/LOOK.
- Buffering, caching, spooling; SSDs need no seek scheduling but have erase-block and wear-leveling constraints.

Common mistakes

- Using `if` instead of `while` around condition variable waits.
- Forgetting that `sem_wait` order matters (deadlock in bounded buffer).
- Counting the initial page loads as hits in replacement traces.
- Leaving the tie-breaking rule unstated in scheduling problems.
- Assuming `fork()` in a loop creates n processes instead of 2^n .

08 Reference: Networking

Layers, addressing, delay math, TCP, and error-detecting codes.

Contents

- Layers
- Addressing and subnetting
- Delay and throughput
- TCP and UDP
- Link layer
- Error detection and correction
- Security basics
- Common mistakes

Layers

OSI	TCP/IP	Unit	Examples
7 Application / 6 Presentation / 5 Session	Application	message	HTTP, DNS, TLS, SSH, MQTT
4 Transport	Transport	segment (TCP) / datagram (UDP)	TCP, UDP, QUIC (over UDP)
3 Network	Internet	packet	IPv4, IPv6, ICMP
2 Data link	Link	frame	Ethernet, Wi-Fi (802.11), ARP
1 Physical	Link	bits	cables, radio, encoding

Encapsulation: each layer adds its header (Ethernet 14 B + 4 B FCS, IPv4 20 B min, TCP 20 B min, UDP 8 B).
Ethernet MTU 1500 B → max TCP payload (MSS) 1460 B without options.

Addressing and subnetting

- IPv4 = 32 bits. Prefix /n: n network bits, 32 – n host bits.
- Addresses per subnet $2^{(32-n)}$. Usable hosts = $2^{(32-n)} - 2$ (network and broadcast reserved) for $n \leq 30$; /31 = 2 (point-to-point links, RFC 3021); /32 = 1.
- Network address = IP AND mask. Broadcast = network OR NOT mask.
- Private ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. Loopback 127.0.0.0/8. Link-local 169.254.0.0/16.
- VLSM: allocate the largest subnets first, each aligned to its own size.
- IPv6 = 128 bits; /64 per LAN; SLAAC; no broadcast (multicast instead).
- MAC address = 48 bits. ARP maps IPv4 → MAC on the local link. Routers forward by longest-prefix match.
- NAT maps private addresses and ports to a public address. DHCP assigns addresses (DORA: Discover, Offer, Request, Ack). DNS resolves names (UDP 53, TCP for large responses).

Calculator: `cecalc.py subnet 192.168.10.0/24 --split 26`, `cecalc.py subnet --hosts 50`.

Delay and throughput

For a packet of L bits on a link of rate R bits/s, length d , propagation speed s :

- transmission delay = L / R
- propagation delay = d / s ($s \approx 2 \times 10^8$ m/s in fiber or copper)
- total per hop = processing + queuing + transmission + propagation
- store-and-forward over N links ($N - 1$ routers), one packet: $N \times L/R$ + total propagation
- bandwidth–delay product = $R \times \text{RTT}$ (bits in flight needed to fill the pipe)
- Stop-and-wait utilization = $(L/R) / (L/R + \text{RTT})$. Sliding window W packets: $\min(1, W \times (L/R) / (L/R + \text{RTT}))$.
- Throughput of a path = the bottleneck link rate.
- Units: 1 Mbps = 10^6 bits/s. 1 MB (file) is usually 10^6 or 2^{20} bytes; state which. Multiply bytes by 8.

Worked example: 1500-byte packet, 100 Mbps link, 2000 km. Transmission = $12\,000 / 10^8 = 120 \mu\text{s}$.

Propagation = $2 \times 10^6 / 2 \times 10^8 = 10$ ms. Propagation dominates.

TCP and UDP

- UDP: connectionless, no reliability, no ordering, 8-byte header. Good for DNS, streaming, games, QUIC.
- TCP: connection-oriented, reliable, ordered byte stream.
 - 3-way handshake: SYN → SYN-ACK → ACK. Teardown: FIN/ACK each direction; TIME_WAIT = $2 \times \text{MSL}$.
 - Sequence numbers count bytes; ACK = next expected byte.
 - **Flow control**: receiver-advertised window (don't overrun the receiver).
 - **Congestion control**: congestion window; slow start (cwnd doubles per RTT until ssthresh), congestion avoidance (+1 MSS per RTT), fast retransmit on 3 duplicate ACKs, fast recovery (Reno: cwnd halves). Timeout → cwnd back to 1 MSS (Tahoe/Reno). CUBIC and BBR are modern defaults.
 - Effective window = $\min(\text{cwnd}, \text{rwnd})$. Throughput $\approx \text{window} / \text{RTT}$.
- Ports: 0–1023 well-known (HTTP 80, HTTPS 443, SSH 22, DNS 53). A connection is identified by the 5-tuple.

Link layer

- Ethernet: CSMA/CD in old half-duplex hubs; switched full-duplex today. Switches learn MAC → port tables. VLANs (802.1Q) split broadcast domains.
- Minimum Ethernet frame 64 bytes (so collisions were detectable in half-duplex).
- Wi-Fi uses CSMA/CA (collision avoidance, ACKs, RTS/CTS for hidden terminals).
- ALOHA efficiency: pure $1/(2e) \approx 18.4\%$, slotted $1/e \approx 36.8\%$.

Error detection and correction

- Parity: detects any odd number of bit errors.
- Internet checksum: 16-bit ones' complement sum of 16-bit words, then complemented. Weak but cheap.
- **CRC**: append r zeros (r = degree of generator), divide mod 2 (XOR), append the remainder. Detects all burst errors of length $\leq r$. Receiver divides the whole frame; remainder 0 means no detected error.
- **Hamming code**: r parity bits for m data bits need $2^r \geq m + r + 1$. Parity bits sit at positions 1, 2, 4, 8, ...; parity bit p covers positions whose index has bit p set. Syndrome = XOR of positions of all 1 bits (with even parity) = position of a single-bit error. SEC-DED adds one overall parity bit to detect (not correct) double errors.

- Hamming distance d : detects $d - 1$ errors, corrects $\lfloor (d - 1)/2 \rfloor$.

Calculator: `cecalc.py crc --data 11010011101100 --poly 1011`, `cecalc.py hamming 1011`, `cecalc.py hamming 0110111 --check`.

Security basics

- Symmetric encryption (AES) for bulk data; asymmetric (RSA, ECC) for key exchange and signatures; hashes (SHA-256) for integrity; MACs (HMAC) for integrity + authenticity.
- TLS: handshake agrees on keys (ECDHE), authenticates the server by certificate, then encrypts with AES-GCM or ChaCha20-Poly1305.
- Firewalls filter by address, port, and connection state.

Common mistakes

- Forgetting the two reserved addresses when counting hosts.
- Mixing bits and bytes in delay calculations.
- Confusing flow control (receiver) with congestion control (network).
- Treating latency and bandwidth as the same thing.
- Using the wrong generator degree when appending zeros for CRC.

09 Reference: Circuits and Electronics

The analog and board-level electronics a computer engineer needs to interface chips safely.

Contents

- Fundamentals
- RC circuits
- Diodes and LEDs
- Transistors as switches
- CMOS logic
- Logic levels and interfacing
- Power and board-level practice
- Measurement tips
- Common mistakes

Fundamentals

- Ohm: $V = I R$. Power: $P = V I = I^2 R = V^2 / R$. Check resistor power rating (1/4 W is common for through-hole, 1/10 W or less for small SMD).
- KCL: currents into a node sum to zero. KVL: voltages around a loop sum to zero.
- Series: $R = R_1 + R_2$. Parallel: $R = R_1 R_2 / (R_1 + R_2)$. Capacitors combine the opposite way.
- Voltage divider: $V_{out} = V_{in} \times R_2 / (R_1 + R_2)$ (unloaded). A load in parallel with R_2 lowers V_{out} ; keep load impedance $\gg R_2$.
- Thevenin: any linear two-terminal network = voltage source + series resistance.
- Units: $\text{mA} \times \text{k}\Omega = \text{V}$. $\mu\text{F} \times \text{k}\Omega = \text{ms}$.

RC circuits

- Time constant $\tau = R C$. Charging: $v(t) = V_{final} (1 - e^{-(t/\tau)})$; 63.2% at τ , 86.5% at 2τ , 95% at 3τ , 99.3% at 5τ .
- 10–90% rise time $\approx 2.2 \tau$.
- First-order low-pass cutoff: $f_c = 1 / (2\pi R C)$; -20 dB/decade above it.
- RC delay on long wires and loaded outputs limits edge rates; capacitive load slows logic.

Calculator: `cecalc.py rc --r 10k --c 100nF --t 1ms`.

Diodes and LEDs

- Silicon diode forward drop $\approx 0.6\text{--}0.7 \text{ V}$; Schottky $\approx 0.2\text{--}0.4 \text{ V}$. LEDs $1.8\text{--}3.3 \text{ V}$ depending on color.
- LED resistor: $R = (V_{supply} - V_f) / I_{LED}$. 3.3 V supply, red LED ($V_f 2.0 \text{ V}$), 10 mA $\rightarrow 130 \Omega$ (use 150 Ω).
- Flyback diode across relay coils and motors (reverse-biased in normal operation) clamps the inductive kick when the switch opens.
- Zener: regulates reverse voltage; TVS diodes clamp ESD and surges.

Transistors as switches

N-channel MOSFET, low side (load between supply and drain, source to ground):

- Turns on when V_{GS} exceeds the threshold by enough. $V_{GS(th)}$ is where it *starts* to conduct (e.g. 250 μA), not fully on. Check $R_{DS(on)}$ at your actual gate voltage.
- With a 3.3 V MCU pin, pick a **logic-level** MOSFET with $R_{DS(on)}$ specified at $V_{GS} = 2.5$ V or 3.3 V (e.g. AO3400-class parts, not IRF540N).
- Add a gate pull-down (10–100 k Ω) so the load stays off while the MCU pin floats during reset, and optionally a small gate resistor (tens of Ω) to limit ringing.
- Power dissipated $\approx I^2 \times R_{DS(on)}$ when fully on.

P-channel MOSFET, high side: turns on when the gate is pulled below the source. Driving from an MCU when the load supply is above the MCU voltage needs an NPN or N-MOSFET level shifter on the gate.

NPN BJT as switch: saturate it. $I_B \geq I_C / \beta_{min} \times (2 \text{ to } 5)$ for margin. $R_B = (V_{pin} - 0.7) / I_B$.
Example: 100 mA load, β_{min} 100, factor 5 $\rightarrow I_B = 5$ mA $\rightarrow R_B = (3.3 - 0.7)/5$ mA $\approx 520 \Omega \rightarrow 470 \Omega$.

CMOS logic

- Inverter: PMOS to V_{DD} , NMOS to ground. NAND: parallel PMOS, series NMOS. NOR: series PMOS, parallel NMOS. NAND is preferred over NOR because series PMOS (slower holes) hurts NOR more.
- Complex gates are naturally inverting (AOI, OAI). A static CMOS gate for function F has pull-down network implementing F' in NMOS.
- **Dynamic power** $P = \alpha C V^2 f$ (α = activity factor). **Static power** from leakage grows as thresholds drop.
- Propagation delay rises with load capacitance and falls with supply voltage.
- Fan-out: how many inputs one output can drive while meeting levels and timing.
- Never leave CMOS inputs floating: they draw shoot-through current and oscillate.

Logic levels and interfacing

Family	V_{OH} min	V_{OL} max	V_{IH} min	V_{IL} max
5 V TTL	2.4	0.4	2.0	0.8
5 V CMOS	4.4	0.5	3.5	1.5
3.3 V LVCMOS	2.4	0.4	2.0	0.8

(Typical values; check the datasheet.)

- Noise margin high = $V_{OH} - V_{IH}$, low = $V_{IL} - V_{OL}$.
- 3.3 V output $\rightarrow$ 5 V TTL input usually works ($2.4 > 2.0$). 3.3 V $\rightarrow$ 5 V CMOS input may not (needs 3.5 V). 5 V $\rightarrow$ 3.3 V input damages a non-5V-tolerant pin; use a divider, a level shifter, or a 5V-tolerant (FT) pin.
- Bidirectional open-drain lines (I2C) shift with a MOSFET-based shifter (BSS138 circuit) or a dedicated IC.

Power and board-level practice

- One 100 nF ceramic decoupling capacitor per power pin, as close to the pin as possible, plus bulk capacitance (1–10 μF) per rail.

- Linear regulator dissipation = $(V_{in} - V_{out}) \times I_{load}$. 12 V $\rightarrow$ 3.3 V at 200 mA wastes 1.74 W: use a buck converter.
- LDO dropout: V_{in} must exceed V_{out} by the dropout voltage.
- Keep ground returns short; a solid ground plane beats thin ground traces. Separate noisy (motor) and quiet (analog) return paths meeting at one point.
- Crystal load capacitors: $C_{load} = (C1 \times C2) / (C1 + C2) + C_{stray}$; with $C1 = C2$, $C1 = 2 (C_{load} - C_{stray})$.
- Signal integrity: when trace length exceeds about 1/6 of the rise-time distance, treat it as a transmission line and terminate it.

Measurement tips

- Multimeter current mode goes in series; voltage in parallel. Never measure voltage in current mode.
- Scope ground clip connects to earth ground on mains-powered scopes: do not clip it to a non-ground node of a mains-referenced circuit.
- Use $\times 10$ probes for fast edges; short ground springs to see real ringing.

Common mistakes

- Using $V_{GS(th)}$ as if it were the full-on gate voltage.
- Forgetting the flyback diode on an inductive load.
- Driving an LED or motor directly from a GPIO pin beyond its current rating.
- Ignoring the load when computing a voltage divider.
- Omitting decoupling capacitors and then chasing "random" resets.

10 Templates

Files in `assets/templates/`.

debug-log.md

```
# Debug Log

**System:** (board, MCU/FPGA, firmware version)
**Date started:** YYYY-MM-DD

## Symptom

What happens, exactly. When. How often. What changed recently.

## Expected behavior

## Known good

What still works, and the last version where this worked.

## Hypotheses

Ranked most to least likely.

| # | Hypothesis | How to test | Result |
|---|---|---|---|
| 1 | | | |
| 2 | | | |
| 3 | | | |

## Measurements

| Time | Probe point / tool | Setting | Observed | Expected |
|---|---|---|---|---|
| | | | | |

## Root cause

## Fix

## How we confirmed it

## Prevent it next time

(test, assertion, checklist item, comment in code)
```

design-review-checklist.md

Design Review Checklist

Project:

Reviewer:

Date: YYYY-MM-DD

Target: (FPGA part / MCU part number / simulator)

Clock(s):

Interface

- [] Every port has a documented width, direction, and meaning
- [] Reset type stated (sync/async, active-high/low) and used consistently
- [] Clock domains listed; every crossing identified

HDL

- [] Sequential logic uses `always\_ff` with `<=`; combinational uses `always\_comb` with `=`
- [] Every `always\_comb` output has a default assignment (no latches in the synthesis report)
- [] Every `case` has a `default`
- [] No width mismatch warnings; constants sized
- [] Signed arithmetic is intentional (`signed` / `\$signed`)
- [] No signal driven from two blocks; no combinational loops
- [] CDC: single bits through 2-FF synchronizers; buses through Gray code, handshake, or async FIFO
- [] Reset deasserts synchronously
- [] Timing constraints written; worst negative slack ≥ 0

Firmware

- [] Peripheral clocks enabled before register access
- [] ISRs short, flags cleared, no blocking calls
- [] ISR-shared data `volatile` and accessed atomically
- [] Stack size checked (per task under an RTOS)
- [] Watchdog refreshed only when all tasks are healthy
- [] Pins in a safe state from reset until configured
- [] Error paths handled (timeouts on every wait loop)

Verification

- [] Self-checking testbench, not waveform eyeballing
- [] Corner cases: reset mid-operation, back-to-back inputs, max/min values, overflow
- [] Tested on hardware at the target clock

Findings

#	Severity	Location	Issue	Fix
---	----------	----------	-------	-----

|---|---|---|---|---|
| 1 | | | | |

formula-sheet.md

Computer Engineering Formula Sheet

Keep only the sections your exam covers.

Numbers

- n-bit unsigned: $0 \dots 2^n - 1$. Two's complement: $-2^{(n-1)} \dots 2^{(n-1)} - 1$
- Negate: invert bits, add 1. Signed overflow: same-sign operands, different-sign result
- IEEE 754 single: 1 | 8 (bias 127) | 23. Double: 1 | 11 (bias 1023) | 52
- Value = $(-1)^s \times 1.f \times 2^{(e - \text{bias})}$

Logic and timing

- De Morgan: $(AB)' = A' + B'$, $(A + B)' = A'B'$
- Setup: $T_{\text{clk}} \geq t_{\text{cq}} + t_{\text{pd}} + t_{\text{setup}}$. Hold: $t_{\text{cq}} + t_{\text{cd}} \geq t_{\text{hold}}$
- $f_{\text{max}} = 1 / (t_{\text{cq}} + t_{\text{pd,max}} + t_{\text{setup}})$
- Dynamic power: $P = \alpha C V^2 f$

Performance

- CPU time = $IC \times CPI / f$
- $CPI = \sum (\text{fraction}_i \times CPI_i)$
- Amdahl: $S = 1 / ((1 - p) + p/s)$
- Pipeline CPI = 1 + stalls per instruction

Caches

- offset = $\log_2(\text{block})$, sets = $\text{size} / (\text{block} \times \text{ways})$, index = $\log_2(\text{sets})$, tag = A - index - offset
- AMAT = hit + miss rate $\times$ penalty (nest for multiple levels, local miss rates)

Virtual memory

- offset = $\log_2(\text{page})$, VPN = VA - offset, flat table = $2^{\text{VPN}} \times \text{PTE size}$
- TLB reach = entries $\times$ page size
- EAT with faults = $(1 - p) \times t_{\text{mem}} + p \times t_{\text{fault}}$

Operating systems

- Turnaround = completion - arrival. Waiting = turnaround - burst
- Deadlock needs: mutual exclusion, hold and wait, no preemption, circular wait
- RMS bound: $\sum C_i/T_i \leq n(2^{(1/n)} - 1)$

Networking

- Transmission = L/R . Propagation = d/s . BDP = $R \times RTT$

- Hosts per subnet = $2^{(32 - \text{prefix})} - 2$
- Hamming parity bits: $2^r \geq m + r + 1$

Embedded and circuits

- Timer period = $\text{prescaler} \times (\text{reload} + 1) / f_{\text{clk}}$
- UART 8N1: $\text{bytes/s} = \text{baud} / 10$
- ADC LSB = $V_{\text{ref}} / 2^N$
- $\tau = RC$, $f_c = 1/(2\pi RC)$, LED $R = (V_s - V_f) / I$

hdl-templates.md

HDL Templates

Starting points for a synthesizable SystemVerilog module and a self-checking testbench. Rename `module\_name`, set the parameters, and replace the placeholder logic.

Contents

- Module template
- Testbench template

Module template

```
// ----- // Module: module_name // Purpose: one
sentence // Clock: clk, rising edge // Reset: rst_n, asynchronous assert, synchronous deassert (external
synchronizer) // Latency: N cycles from in_valid to out_valid // -----
----- module module_name #( parameter int W = 8 ) ( input logic clk, input logic rst_n, input logic
in_valid, input logic [W-1:0] in_data, output logic out_valid, output logic [W-1:0] out_data );

// ----- state logic [W-1:0] data_q, data_d; logic valid_q;

// ----- next-state logic always_comb begin data_d = data_q;
// defaults first: no latches if (in_valid) data_d = in_data; end

// ----- registers always_ff @(posedge clk or negedge rst_n)
begin if (!rst_n) begin data_q <= '0; valid_q <= 1'b0; end else begin data_q <= data_d; valid_q <= in_valid; end
end

// ----- outputs assign out_data = data_q; assign out_valid =
valid_q;

endmodule
```

Testbench template

```
// Self-checking testbench for module_name. Prints PASS/FAIL and a summary. `timescale 1ns/1ps module
module_name_tb; localparam int W = 8; localparam time CLK_PERIOD = 10ns;
```

```

logic clk = 0, rst_n = 0; logic in_valid = 0; logic [W-1:0] in_data = '0; logic out_valid; logic [W-1:0] out_data; int
errors = 0;

module_name #(.W(W)) dut (.*);

always #(CLK_PERIOD/2) clk = ~clk;

// Drive one input, then check the output one cycle later. task automatic apply_and_check(input logic [W-1:0]
value); @(negedge clk); in_valid = 1; in_data = value; @(negedge clk); in_valid = 0; if (!out_valid || out_data !==
value) begin $error("value %0h: got valid=%0b data=%0h", value, out_valid, out_data); errors++; end endtask

initial begin repeat (3) @(negedge clk); rst_n = 1;

apply_and_check('0); // minimum apply_and_check('1); // maximum apply_and_check(W'(8'hA5)); // pattern
repeat (20) apply_and_check(W'($urandom)); // random

if (errors == 0) $display("PASS"); else $display("FAIL: %0d errors", errors); $finish; end endmodule

```

lab-report.md

Lab Report: TITLE

Course:

Lab number:

Name(s):

Date: YYYY-MM-DD

Objective

One or two sentences: what you built or measured, and why.

Background

The concepts and equations the lab relies on. Define every symbol.

Design

- Block diagram or schematic
- Interface: inputs, outputs, widths, clock and reset
- Key design decisions and the reason for each

Procedure

Numbered steps someone else could repeat.

Results

Test	Input	Expected	Measured	Pass?	
---	---	---	---	---	

Include waveforms, simulation screenshots, or scope captures with labeled axes.

Analysis

- Compare measured against expected; explain every difference
- Error sources and their size
- Timing: critical path, max frequency, resource use (LUTs, FFs, memory)

Conclusion

What worked, what didn't, what you'd change.

Appendix

Source code, constraints files, raw data.

11 Appendix: cecalc.py Source

The complete calculator. Save it as `compenclaude/scripts/cecalc.py`.

```
#!/usr/bin/env python3
"""cecalc: exact calculators for computer engineering problems.

Standard library only (Python 3.9+). Every command prints its working, so the
numbers can be checked by hand.

python3 cecalc.py <command> -h      # options for one command

Commands
conv      integer bases, two's complement, Gray code, byte order
float     IEEE 754 encode/decode (single or double)
cache     address breakdown + optional trace simulation (LRU/FIFO, 3C misses)
amat      average memory access time, multi-level
cpu        iron law: CPU time from IC, CPI (or instruction mix), clock
amdahl    overall speedup, or speedup needed to hit a target
vm         virtual memory: page offset/VPN bits, page-table size, levels, TLB reach
subnet    IPv4/IPv6 subnet facts, splitting, prefix for N hosts
timer     MCU timer prescaler/reload search for a target frequency
baud       UART divisor and baud error
adc        ADC LSB size and code <=> voltage
rc         RC time constant, cutoff frequency, charge fraction
hamming    Hamming(SEC) encode or check/correct
crc        CRC remainder by mod-2 long division
"""
from __future__ import annotations

import argparse
import ipaddress
import math
import re
import struct
import sys
from collections import OrderedDict
from decimal import Decimal, InvalidOperation, getcontext

getcontext().prec = 80

# ----- parsing

_SI = {
    "": 1.0, "p": 1e-12, "n": 1e-9, "u": 1e-6, "μ": 1e-6, "m": 1e-3,
    "k": 1e3, "K": 1e3, "M": 1e6, "G": 1e9, "T": 1e12
}
_SI_RE = re.compile(
    r"^\s*([+-]?(\?:\d+\.\d*|\.\d+)(?:[eE] [+-]?\d+)?)\s*([pnuμmKMG]?)\s*"

```

```

r"(?i:hz|ohms?|\Q|f|s|v|a|bps|baud)?\s*$")
_SIZE_RE = re.compile(r"^\\s*(\\d+)\\s*([kKmMgGtT]?)(?:i?[bB])?\\s*$")

def die(msg: str) -> None:
    sys.exit(f"error: {msg}")

def parse_si(text: str) -> float:
    """'16MHz' -> 16e6, '100n' -> 1e-7, '4.7uF' -> 4.7e-6, '2e6' -> 2e6."""
    m = _SI_RE.match(text)
    if not m:
        die(f"cannot parse quantity {text!r} (try forms like 16MHz, 10k, 100n, 2e6)")
    return float(m.group(1)) * _SI[m.group(2)]

def parse_size(text: str) -> int:
    """Byte sizes are binary: '32KiB', '32KB' and '32K' all mean 32768."""
    m = _SIZE_RE.match(text)
    if not m:
        return parse_int(text)
    return int(m.group(1)) * 1024 ** "_KMG".index(m.group(2).upper() or "_")

def parse_int(text: str) -> int:
    t = text.strip().replace("_", "")
    try:
        return int(t, 0)
    except ValueError:
        try:
            return int(t, 10)
        except ValueError:
            die(f"cannot parse integer {text!r} (use 42, -7, 0x2A, 0b101010, 0o52)")
    raise AssertionError

def ilog2(n: int, what: str) -> int:
    if n <= 0 or n & (n - 1):
        die(f"{what} must be a power of two (got {n})")
    return n.bit_length() - 1

# ----- formatting

def eng(x: float, unit: str = "") -> str:
    if x == 0 or math.isinf(x) or math.isnan(x):
        return f"{x:g} {unit}".strip()
    for exp, p in ((12, "T"), (9, "G"), (6, "M"), (3, "k"), (0, "")),

```

```

        (-3, "m"), (-6, "μ"), (-9, "n"), (-12, "p"))):
    if abs(x) >= 10 ** exp:
        return f"{x / 10 ** exp:.6g} {p}{unit}".strip()
    return f"{x:.6g} {unit}".strip()

def human_bytes(n: int) -> str:
    for p, unit in ((4, "TiB"), (3, "GiB"), (2, "MiB"), (1, "KiB")):
        if n >= 1024 ** p and n % 1024 ** p == 0:
            return f"{n // 1024 ** p} {unit} ({n} B)"
    return f"{n} B"

def grouped_bin(v: int, bits: int) -> str:
    s = format(v, f"0{bits}b")
    head = len(s) % 4
    parts = ([s[:head]] if head else []) + [s[i:i + 4] for i in range(head, len(s), 4)]
    return "_".join(parts)

def row(label: str, value) -> None:
    print(f" {label:<24}{value}")

# ----- conv

def cmd_conv(a) -> None:
    v = parse_int(a.value)
    bits = a.bits
    if bits is None:
        for bits in (8, 16, 32, 64, 128, 256):
            if (v >= 0 and v < 1 << bits) or (v < 0 and v >= -(1 << (bits - 1))):
                break
    if bits <= 0:
        die("--bits must be positive")
    if v < 0:
        if v < -(1 << (bits - 1)):
            die(f"{v} does not fit in {bits}-bit two's complement "
                f"(range {-(1 << (bits - 1))} .. {(1 << (bits - 1)) - 1})")
    elif v >= 1 << bits:
        die(f"{v} does not fit in {bits} bits (max unsigned {(1 << bits) - 1})")
    pattern = v & ((1 << bits) - 1)
    signed = pattern - (1 << bits) if pattern >> (bits - 1) else pattern
    hexw = (bits + 3) // 4

    print(f"{a.value} as a {bits}-bit pattern")
    row("hex", f"0x{pattern:0{hexw}X}")
    row("binary", grouped_bin(pattern, bits))

```

```

row("octal", f"0o{pattern:o}")
row("unsigned", pattern)
row("signed (two's compl.)", signed)
row("unsigned range", f"0 .. {(1 << bits) - 1}")
row("signed range", f"~{(1 << (bits - 1))} .. {(1 << (bits - 1)) - 1}")
row("Gray code (b ^ b>>1)", grouped_bin(pattern ^ (pattern >> 1), bits))
row("popcount", bin(pattern).count("1"))
if bits % 8 == 0:
    be = pattern.to_bytes(bits // 8, "big")
    row("memory, big-endian", " ".join(f"{b:02X}" for b in be))
    row("memory, little-endian", " ".join(f"{b:02X}" for b in reversed(be)))
if v >= 0 and signed < 0:
    print(f" note: MSB is set, so the same bits read as {signed} when signed.")

# ----- float

_FMT = {32: (">f", ">I", 8, 23, 127), 64: (">d", ">Q", 11, 52, 1023)}

def cmd_float(a) -> None:
    note = None
    if a.hex:
        raw = parse_int(a.value)
        digits = len(a.value.lower().replace("0x", "").replace("_", ""))
        width = 64 if a.double or digits > 8 else 32
        if raw >> width:
            die(f"pattern does not fit in {width} bits")
        ffmt, ifmt, eb, mb, bias = _FMT[width]
        stored = struct.unpack(ffmt, struct.pack(ifmt, raw))[0]
    else:
        width = 64 if a.double else 32
        ffmt, ifmt, eb, mb, bias = _FMT[width]
        try:
            x = float(a.value)
        except ValueError:
            die(f"cannot parse float {a.value!r}")
        try:
            raw = struct.unpack(ifmt, struct.pack(ffmt, x))[0]
        except OverflowError:
            raw = ((1 if x < 0 else 0) << (eb + mb)) | (((1 << eb) - 1) << mb)
            note = "magnitude exceeds the largest finite value: rounds to infinity"
        stored = struct.unpack(ffmt, struct.pack(ifmt, raw))[0]

    sign = raw >> (eb + mb)
    exp = (raw >> mb) & ((1 << eb) - 1)
    frac = raw & ((1 << mb) - 1)
    name = "single (binary32)" if width == 32 else "double (binary64)"

```

```

print(f"IEEE 754 {name}: 1 sign, {eb} exponent (bias {bias}), {mb} fraction bits")
row("hex", f"0x{raw:0{width // 4}X}")
row("fields s|exp|frac", f"{sign} | {exp:0{eb}b} | {frac:0{mb}b}")

if exp == (1 << eb) - 1:
    row("class", "NaN" if frac else ("-inf" if sign else "+inf"))
elif exp == 0 and frac == 0:
    row("class", "-0" if sign else "+0")
else:
    if exp == 0:
        e, lead, cls = 1 - bias, "0", "subnormal"
    else:
        e, lead, cls = exp - bias, "1", "normal"
    row("class", cls)
    row("exponent", f"stored {exp}, unbiased {e}")
    mant = f"{frac:0{mb}b}".rstrip("0") or "0"
    row("value", f"{'-' if sign else '+'}{lead}.{mant} (binary) x 2^{e}")
    exact = f"{Decimal(stored):f}"
    if "." in exact:
        exact = exact.rstrip("0").rstrip(".")
    row("exact stored value", exact if abs(e) < 60 else f"{Decimal(stored):E}")
    row("shortest repr", repr(stored))
    row("ULP at this value", f"2^{e - mb} = {2.0 ** (e - mb):.6g}")
    if not a.hex:
        try:
            err = Decimal(a.value) - Decimal(stored)
            row("rounding error", "0 (exact)" if err == 0 else f"{err:.6E}")
        except InvalidOperation:
            pass
    if note:
        print(f"  note: {note}")

```

----- cache

```

def _read_trace(a) -> list[int]:
    addrs = [parse_int(t) for t in (a.trace or [])]
    if a.trace_file:
        with open(a.trace_file) as fh:
            for line in fh:
                tokens = line.split("#")[0].split()
                if tokens:
                    addrs.append(parse_int(tokens[-1])) # tolerate "R 0x1f" / "W 0x20"
    return addrs

```

```

def cmd_cache(a) -> None:
    size, block = parse_size(a.size), parse_size(a.block)

```

```

if size % block:
    die("cache size must be a multiple of block size")
lines = size // block
ways = lines if str(a.assoc).lower() in ("full", "fa") else parse_int(a.assoc)
if ways < 1 or lines % ways:
    die(f"associativity {ways} does not divide {lines} lines")
sets = lines // ways
ob, ib = ilog2(block, "block size"), ilog2(sets, "number of sets")
tb = a.addr_bits - ib - ob
if tb <= 0:
    die("address too short for this geometry")

kind = "direct-mapped" if ways == 1 else ("fully associative" if sets == 1 else f"{ways}-way set
associative")
print(f"{human_bytes(size)} {kind}, {block} B blocks, {a.addr_bits}-bit byte addresses")
row("lines (blocks)", lines)
row("sets x ways", f"{sets} x {ways}")
row("offset bits", f"{ob} addr[{ob - 1}:0]" if ob else "0")
row("index bits", f"{ib} addr[{ib + ob - 1}:{ob}]" if ib else "0 (fully associative)")
row("tag bits", f"{tb} addr[{a.addr_bits - 1}:{ib + ob}]")
meta = tb + 1 + (1 if a.write_back else 0)
row("metadata per line", f"{tb} tag + 1 valid" + (" + 1 dirty" if a.write_back else "")
    + f" = {meta} bits")
row("total storage", f"{size * 8} data + {lines * meta} metadata = {size * 8 + lines * meta} bits")

addrs = _read_trace(a)
if not addrs:
    return

print(f"\nTrace ({a.policy.upper()} replacement, allocate on every miss)")
print(f" {'#':>3} {'address':>12} {'tag':>10} {'set':>5} {'off':>4} result {'evicted tag':>11}
miss type")
cache = [OrderedDict() for _ in range(sets)]
shadow: OrderedDict[int, None] = OrderedDict() # fully-assoc LRU, same capacity
seen: set[int] = set()
counts = {"hit": 0, "compulsory": 0, "capacity": 0, "conflict": 0}
for n, addr in enumerate(addrs, 1):
    blk = addr >> ob
    off, idx, tag = addr & (block - 1), blk & (sets - 1), blk >> ib
    s = cache[idx]
    shadow_hit = blk in shadow
    if shadow_hit:
        shadow.move_to_end(blk)
    else:
        if len(shadow) == lines:
            shadow.popitem(last=False)
        shadow[blk] = None
    evicted = ""

```

```

if tag in s:
    result, mtype = "hit ", ""
    counts["hit"] += 1
    if a.policy == "lru":
        s.move_to_end(tag)
else:
    result = "MISS"
    mtype = "compulsory" if blk not in seen else ("conflict" if shadow_hit else "capacity")
    counts[mtype] += 1
    if len(s) == ways:
        evicted = f"0x{s.popitem(last=False)[0]:X}"
    s[tag] = None
    seen.add(blk)
    print(f"  {n:>3}  {'0x%X' % addr:>12}  {'0x%X' % tag:>10}  {idx:>5}  {off:>4}  {result}
{evicted:>11}  {mtype}")
    total = len(addr)
    misses = total - counts["hit"]
    print(f"\n hits {counts['hit']}/{total} ({counts['hit'] / total:.2%}), misses {misses} "
        f"(compulsory {counts['compulsory']}, capacity {counts['capacity']}, conflict
{counts['conflict']})")

# ----- amat / cpu / amdahl

def cmd_amat(a) -> None:
    levels = []
    for spec in a.levels:
        try:
            hit, mr = spec.split(":")
            levels.append((float(hit), float(mr)))
        except ValueError:
            die(f"level {spec!r} must look like HIT_TIME:LOCAL_MISS_RATE, e.g. 1:0.05")
    penalty = a.mem
    terms = []
    for i in range(len(levels) - 1, -1, -1):
        hit, mr = levels[i]
        terms.append(f"L{i + 1}: {hit:g} + {mr:g} x {penalty:g} = {hit + mr * penalty:g}")
        penalty = hit + mr * penalty
    print("AMAT = hit time + local miss rate x miss penalty (innermost first)")
    for t in reversed(terms):
        row("", t)
    row("AMAT", f"{penalty:g} (same unit as inputs)")
    g = 1.0
    for i, (_, mr) in enumerate(levels):
        g *= mr
        row(f"global miss rate L{i + 1}", f"{g:.6g}")

```

```

def cmd_cpu(a) -> None:
    ic = parse_si(a.ic)
    if a.mix:
        cpi, parts = 0.0, []
        total_frac = 0.0
        for spec in a.mix.split(","):
            f, c = (float(x) for x in spec.split(":"))
            total_frac += f
            cpi += f * c
            parts.append(f"{f:g}x{c:g}")
        print(f"CPI = {' + '.join(parts)} = {cpi:g}")
        if abs(total_frac - 1) > 1e-9:
            print(f" warning: fractions sum to {total_frac:g}, not 1")
    elif a.cpi is not None:
        cpi = a.cpi
    else:
        die("give --cpi or --mix")
    if a.clock:
        f = parse_si(a.clock)
    elif a.period:
        f = 1 / parse_si(a.period)
    else:
        die("give --clock or --period")
    t = ic * cpi / f
    print("CPU time = IC x CPI / f")
    row("instruction count", f"{ic:g}")
    row("CPI / IPC", f"{cpi:g} / {1 / cpi:.6g}")
    row("clock", f"{eng(f, 'Hz')} (period {eng(1 / f, 's')})")
    row("cycles", f"{ic * cpi:g}")
    row("CPU time", eng(t, "s"))
    row("native MIPS", f"{f / (cpi * 1e6):.6g}")

def cmd_amdahl(a) -> None:
    p = a.fraction
    if not 0 <= p <= 1:
        die("--fraction must be between 0 and 1")
    print("Speedup = 1 / ((1 - p) + p / s)")
    row("limit (s -> inf)", "inf" if p == 1 else f"{1 / (1 - p):.6g}x")
    if a.speedup is not None:
        s = a.speedup
        overall = 1 / ((1 - p) + (0 if math.isinf(s) else p / s))
        row(f"overall at s={s:g}", f"{overall:.6g}x")
    if a.target is not None:
        denom = 1 / a.target - (1 - p)
        if denom <= 0:
            row(f"s for {a.target:g}x", "impossible: target exceeds the limit")
        else:

```

```

        row(f"s for {a.target:g}x", f"{p / denom:.6g}x on the improved fraction")

# ----- vm

def cmd_vm(a) -> None:
    page = parse_size(a.page)
    ob = ilog2(page, "page size")
    vpn = a.va_bits - ob
    print(f"{a.va_bits}-bit virtual addresses, {human_bytes(page)} pages, {a.pte}-byte PTEs")
    row("page offset bits", ob)
    row("VPN bits", vpn)
    row("flat table", f"2^{vpn} entries x {a.pte} B = {human_bytes((1 << vpn) * a.pte)}")
    if a.pa_bits:
        row("PFN bits", a.pa_bits - ob)
        row("physical pages", f"2^{a.pa_bits - ob}")
    per = page // a.pte
    bpl = ilog2(per, "PTEs per page")
    levels = math.ceil(vpn / bpl)
    row("PTEs per page", f"{per} ({bpl} VPN bits per level)")
    row("levels (table = 1 page)", f"{levels} split " +
        "+".join(str(min(bpl, vpn - bpl * i)) for i in reversed(range(levels))) + f" + {ob} offset")
    if a.tlb:
        row("TLB reach", human_bytes(a.tlb * page))

# ----- subnet

def cmd_subnet(a) -> None:
    if a.hosts is not None:
        need = a.hosts + 2
        prefix = 32 - math.ceil(math.log2(need)) if a.hosts > 2 else (31 if a.hosts == 2 else 32)
        print(f"{a.hosts} hosts need /{prefix} "
            f"({2 ** (32 - prefix)} addresses, {max(2 ** (32 - prefix) - 2, a.hosts)} usable)")
        if not a.cidr:
            return
    if not a.cidr:
        die("give a CIDR block (e.g. 192.168.1.0/26) or --hosts N")
    try:
        net = ipaddress.ip_network(a.cidr, strict=False)
    except ValueError as e:
        die(str(e))
    print(str(net) + (f" (normalized from {a.cidr})" if str(net) != a.cidr else ""))
    row("netmask", net.netmask)
    if net.version == 4:
        row("wildcard", net.hostmask)
    row("network", net.network_address)
    if net.version == 4:

```

```

        row("broadcast", net.broadcast_address)
total = net.num_addresses
if net.version == 4 and net.prefixlen <= 30:
    usable, first, last = total - 2, net.network_address + 1, net.broadcast_address - 1
else:
    usable, first, last = total, net.network_address, net.broadcast_address
row("total addresses", total)
row("usable hosts", usable)
row("host range", f"{first} - {last}")
row("private (RFC1918 etc.)", net.is_private)
if a.split:
    if a.split <= net.prefixlen:
        die("--split prefix must be longer than the network prefix")
    subs = list(net.subnets(new_prefix=a.split))
    print(f"\n {len(subs)} subnets of /{a.split}:")
    for s in subs[:64]:
        print(f"    {s}")
    if len(subs) > 64:
        print(f"    ... {len(subs) - 64} more")

# ----- embedded

def cmd_timer(a) -> None:
    clk = parse_si(a.clock)
    target = 1 / parse_si(a.period) if a.period else parse_si(a.target)
    prescalers = [int(x) for x in a.prescalers.split(",")]
    top = 1 << a.bits
    print(f"f_clk {eng(clk, 'Hz')}, target {eng(target, 'Hz')} ({eng(1 / target, 's')}), {a.bits}-bit
counter")
    print(" counts = f_clk / (prescaler x f_target); counter runs 0..reload, so reload = counts - 1")
    print(f" {'prescaler':>9} {'counts':>10} {'reload':>8} {'actual':>14} {'error':>9}")
    best = None
    for p in prescalers:
        exact = clk / (p * target)
        n = round(exact)
        if n < 1 or n > top:
            print(f" {p:>9} {exact:>10.2f} {'out of range':>8}")
            continue
        actual = clk / (p * n)
        err = (actual - target) / target
        print(f" {p:>9} {n:>10} {n - 1:>8} {eng(actual, 'Hz'):>14} {err:>+9.4%}")
        if best is None or abs(err) < abs(best[1]) - 1e-15:
            best = (p, err, n)
    if best:
        print(f"\n best: prescaler {best[0]}, reload {best[2] - 1} "
            f"(STM32: PSC={best[0] - 1}, ARR={best[2] - 1}; AVR CTC: OCRnA={best[2] - 1})")
    else:

```

```

        print("\n no prescaler reaches the target; use a wider timer or a larger prescaler")

def cmd_baud(a) -> None:
    clk, baud, os_ = parse_si(a.clock), parse_si(a.baud), a.oversample
    exact = clk / (os_ * baud)
    n = max(1, round(exact))
    actual = clk / (os_ * n)
    err = (actual - baud) / baud
    print(f"f_clk {eng(clk, 'Hz')}, target {baud:g} baud, oversampling x{os_}")
    row("divisor", f"{exact:.4f} -> {n}")
    row("actual baud", f"{actual:.2f}")
    row("error", f"{err:+.3%}" + ("    WARNING: above 2%, link may be unreliable" if abs(err) > 0.02 else
""))
    row("AVR UBRR (U2X=0)", n - 1 if os_ == 16 else "n/a (use --oversample 16)")
    if os_ == 8:
        row("AVR UBRR (U2X=1)", n - 1)
    row("STM32 BRR (OVER8=0)", f"{round(clk / baud)} (fractional divider, error "
f"{{(clk / round(clk / baud) - baud) / baud:+.3%}})")

def cmd_adc(a) -> None:
    vref = parse_si(a.vref)
    levels = 1 << a.bits
    lsb = vref / levels
    print(f"{a.bits}-bit ADC, Vref {eng(vref, 'V')}")
    row("codes", f"0 .. {levels - 1}")
    row("LSB = Vref / 2^N", eng(lsb, "V"))
    row("alt. Vref / (2^N - 1)", eng(vref / (levels - 1), "V") + " (some datasheets)")
    if a.code is not None:
        c = parse_int(a.code)
        if not 0 <= c < levels:
            die(f"code must be 0..{levels - 1}")
        row(f"code {c} ->", f"{eng(c * lsb, 'V')} (bin low edge), {eng((c + 0.5) * lsb, 'V')} (bin
centre)")
    if a.volts is not None:
        v = parse_si(a.volts)
        c = min(levels - 1, max(0, math.floor(v / lsb)))
        row(f"{eng(v, 'V')} ->", f"code {c} (0x{c:X})")
    row("quantization error", f"±{eng(lsb / 2, 'V')} (ideal, rounding ADC)")
    row("ideal SNR", f"{6.02 * a.bits + 1.76:.2f} dB (full-scale sine)")

def cmd_rc(a) -> None:
    r, c = parse_si(a.r), parse_si(a.c)
    tau = r * c
    print(f"R = {eng(r, 'Ω')}, C = {eng(c, 'F')}")
    row("tau = RC", eng(tau, "s"))

```

```

row("-3 dB cutoff  $1/(2\pi RC)$ ", eng(1 / (2 * math.pi * tau), "Hz"))
row("10-90% rise (2.2 tau)", eng(2.2 * tau, "s"))
row("settle to 99.3% (5 tau)", eng(5 * tau, "s"))
if a.t:
    t = parse_si(a.t)
    frac = 1 - math.exp(-t / tau)
    row(f"charge after {eng(t, 's')}", f"{frac:.4%} of final value")

# ----- coding

def _bits(text: str, what: str) -> list[int]:
    t = text.replace("_", "").replace(" ", "")
    if not t or set(t) - {"0", "1"}:
        die(f"{what} must be a string of 0s and 1s")
    return [int(ch) for ch in t]

def cmd_hamming(a) -> None:
    if a.check:
        code = _bits(a.value, "codeword")
        n = len(code)
        syndrome = 0
        for pos in range(1, n + 1):
            if code[pos - 1]:
                syndrome ^= pos
        print(f"Hamming check of {a.value} (positions 1..{n}, left to right, even parity)")
        row("syndrome", f"{syndrome} (0b{syndrome:b})")
        if syndrome == 0:
            row("result", "no single-bit error detected")
        elif syndrome > n:
            row("result", "syndrome points past the word: multi-bit error")
        else:
            code[syndrome - 1] ^= 1
            row("result", f"bit {syndrome} flipped; corrected {''.join(map(str, code))}")
            data = [code[p - 1] for p in range(1, n + 1) if p & (p - 1)]
            row("data bits", "".join(map(str, data)))
            print(" note: a single syndrome cannot tell a 2-bit error from a 1-bit one; use SEC-DED for that.")
        return
    data = _bits(a.value, "data")
    m = len(data)
    r = 0
    while (1 << r) < m + r + 1:
        r += 1
    n = m + r
    code = [0] * (n + 1) # 1-indexed
    it = iter(data)

```

```

for pos in range(1, n + 1):
    if pos & (pos - 1):
        code[pos] = next(it)
print(f"Hamming({n},{m}) encode of {a.value}: parity at positions 1,2,4,...; even parity")
for i in range(r):
    p = 1 << i
    covered = [q for q in range(1, n + 1) if q & p and q != p]
    code[p] = sum(code[q] for q in covered) % 2
    row(f"p{p}", f"XOR of positions {covered} = {code[p]}")
row("codeword (pos 1..n)", "".join(map(str, code[1:])))

```

```

def cmd_crc(a) -> None:
    data, poly = _bits(a.data, "data"), _bits(a.poly, "generator")
    if poly[0] != 1:
        die("generator must start with 1")
    deg = len(poly) - 1
    work = data + [0] * deg
    print(f"CRC: data {a.data}, generator {a.poly} (degree {deg}); append {deg} zeros and divide mod 2")
    for i in range(len(data)):
        if work[i]:
            for j in range(len(poly)):
                work[i + j] ^= poly[j]
    rem = work[-deg:] if deg else []
    row("remainder", "".join(map(str, rem)) or "(none)")
    row("transmitted", "".join(map(str, data + rem)))

```

----- CLI

```

def main() -> None:
    ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    sp = ap.add_subparsers(dest="cmd", required=True)

    p = sp.add_parser("conv", help="integer bases / two's complement")
    p.add_argument("value", help="42, -7, 0x2A, 0b101010, 0o52")
    p.add_argument("--bits", type=int, help="bit width (default: smallest of 8/16/32/64 that fits)")
    p.set_defaults(fn=cmd_conv)

    p = sp.add_parser("float", help="IEEE 754 encode/decode")
    p.add_argument("value", help="decimal number, or a bit pattern with --hex")
    p.add_argument("--hex", action="store_true", help="decode VALUE as a raw bit pattern")
    p.add_argument("--double", action="store_true", help="use binary64")
    p.set_defaults(fn=cmd_float)

    p = sp.add_parser("cache", help="cache geometry and trace simulation")
    p.add_argument("--size", required=True, help="data capacity, e.g. 32KiB")
    p.add_argument("--block", required=True, help="block/line size in bytes, e.g. 64")

```

```

p.add_argument("--assoc", default="1", help="ways (1 = direct-mapped) or 'full'")
p.add_argument("--addr-bits", type=int, default=32)
p.add_argument("--write-back", action="store_true", help="count a dirty bit per line")
p.add_argument("--policy", choices=["lru", "fifo"], default="lru")
p.add_argument("--trace", nargs="*", help="byte addresses to simulate")
p.add_argument("--trace-file", help="file with one address per line (last token used)")
p.set_defaults(fn=cmd_cache)

p = sp.add_parser("amat", help="average memory access time")
p.add_argument("--levels", nargs="+", required=True, help="HIT:LOCAL_MISS_RATE per level, L1 first")
p.add_argument("--mem", type=float, required=True, help="main-memory access time")
p.set_defaults(fn=cmd_amat)

p = sp.add_parser("cpu", help="iron law of performance")
p.add_argument("--ic", required=True, help="instruction count, e.g. 2e9 or 2G")
p.add_argument("--cpi", type=float)
p.add_argument("--mix", help="FRACTION:CPI pairs, e.g. 0.5:1,0.3:2,0.2:5")
p.add_argument("--clock", help="e.g. 3GHz")
p.add_argument("--period", help="e.g. 250ps")
p.set_defaults(fn=cmd_cpu)

p = sp.add_parser("amdahl", help="Amdahl's law")
p.add_argument("--fraction", type=float, required=True, help="fraction of time that is improved")
p.add_argument("--speedup", type=float, help="speedup of that fraction (inf allowed)")
p.add_argument("--target", type=float, help="solve for the speedup needed to reach this overall")
p.set_defaults(fn=cmd_amdahl)

p = sp.add_parser("vm", help="paging arithmetic")
p.add_argument("--va-bits", type=int, required=True)
p.add_argument("--page", required=True, help="page size, e.g. 4KiB")
p.add_argument("--pte", type=int, default=8, help="bytes per page-table entry")
p.add_argument("--pa-bits", type=int)
p.add_argument("--tlb", type=int, help="TLB entries, for reach")
p.set_defaults(fn=cmd_vm)

p = sp.add_parser("subnet", help="IP subnetting")
p.add_argument("cidr", nargs="?", help="e.g. 10.0.0.0/22")
p.add_argument("--split", type=int, help="list subnets of this longer prefix")
p.add_argument("--hosts", type=int, help="smallest IPv4 prefix holding N hosts")
p.set_defaults(fn=cmd_subnet)

p = sp.add_parser("timer", help="MCU timer prescaler/reload")
p.add_argument("--clock", required=True, help="timer input clock, e.g. 16MHz")
p.add_argument("--target", help="interrupt/overflow frequency, e.g. 1kHz")
p.add_argument("--period", help="or the period, e.g. 10ms")
p.add_argument("--bits", type=int, default=16)
p.add_argument("--prescalers", default="1,8,64,256,1024", help="comma list")
p.set_defaults(fn=cmd_timer)

```

```
p = sp.add_parser("baud", help="UART divisor and error")
p.add_argument("--clock", required=True)
p.add_argument("--baud", required=True)
p.add_argument("--oversample", type=int, default=16)
p.set_defaults(fn=cmd_baud)
```

```
p = sp.add_parser("adc", help="ADC resolution and conversion")
p.add_argument("--bits", type=int, required=True)
p.add_argument("--vref", required=True, help="e.g. 3.3V")
p.add_argument("--code", help="convert a code to volts")
p.add_argument("--volts", help="convert a voltage to a code")
p.set_defaults(fn=cmd_adc)
```

```
p = sp.add_parser("rc", help="RC circuit timing")
p.add_argument("--r", required=True, help="e.g. 10k")
p.add_argument("--c", required=True, help="e.g. 100nF")
p.add_argument("--t", help="elapsed time for charge fraction, e.g. 1ms")
p.set_defaults(fn=cmd_rc)
```

```
p = sp.add_parser("hamming", help="Hamming code encode/check")
p.add_argument("value", help="data bits (encode) or codeword (with --check)")
p.add_argument("--check", action="store_true")
p.set_defaults(fn=cmd_hamming)
```

```
p = sp.add_parser("crc", help="CRC by polynomial long division")
p.add_argument("--data", required=True, help="message bits, e.g. 11010011101100")
p.add_argument("--poly", required=True, help="generator bits, e.g. 1011 for  $x^3+x+1$ ")
p.set_defaults(fn=cmd_crc)
```

```
args = ap.parse_args()
args.fn(args)
```

```
if __name__ == "__main__":
    main()
```